PA0 - 世界诞生的前夜：开发环境配置

实验进度

我完成了所有安装任务，完成了必答题，并针对思考题和遇到的其他问题进行了资料搜索和思考。

必答题

我按照实验要求编写了 hello.c 并编写 Makefile 文件进行编译，并且学习了 GDB 的使用。这些内容无法直接呈现，故省略。

关于《提问的智慧》

我们从小被要求养成提问的习惯，提问被认为是有好奇心或者善于探索的象征，但我认为背后有思考的提问才是真正有价值的。《提问的智慧》中提及了大量的提问相关的 convention 和技巧，其核心原理都是思考。

我曾在知乎上看到一句话：“道德是用来约束自己的，不是用来约束别人的。”这句话当时被用来说明你不能强求别人写给你看的题解像喂饭一样深入浅出。我觉得这句话用在提问上也一样合适。除非你花钱雇了一个人专职为你解答问题，否则别人解答你的问题是一种义务劳动。你没有权利浪费别人的时间，别人也没有义务去回答一些愚蠢的问题或者给你保姆级的教程。我总结了一些《提问的智慧》中我觉得比较精华的内容罗列如下：

- **不要问一些直接 STFW/RTFM 就能轻松获得答案的知识性问题**，他们通常具有 "xxx是什么？""xxx怎么用” 等格式。
- 作为一个计算机界的后辈，你遇到的问题有极大的概率是已经有无数人遇到过的问题。一些经典的问题通常可以在 Stack Overflow 等知名平台上找到解答，所以**你应该确保你的问题没有现成可查的解决方案**。
- 你应该**尽可能详细地刻画你的问题**，比如写下你已经用过哪些方法但失败了，也可以贴一些截图。一方面，更详细的信息可以让别人更好地了解你的情况，提供帮助；另一方面，这可以体现你为了解决问题已经付出过你能力范围内的努力。
- **机器永远是对的**，不要说“我觉得我的gcc有问题/我觉得我的电脑坏了”之类的话（这是我在帮助上“程序设计基础”课的同学解决问题时经常听到的话）。另外，你手上的源码有很大概率经过了很多人的验证，所以**不要轻易宣布自己发现了 bug**。
- 作为一个提问者你不能显得趾高气昂，在提问中**适当使用委婉的词汇**可以使你的态度更加诚恳，但也不用过度使用以至于语句冗余。
- 对于我们这样的初学者来说问题可能还没有领域之分，但在日后如果提出一些分类较细的问题，需要注意提问的平台是否合适。

总之，想要提出一个好的问题，你需要在遇到困难时先主动自己解决，尝试能力范围内可以使用的方法，并且在你的提问中展现这个过程。这样蕴含了你的思考的问题才是有价值的。

以下是我的自由报告内容，我在做实验的过程中详细记录了自己遇到的所有问题和查阅的所有资料。这些笔记按照章节排列，每章通常有两个部分：

- 思考题：包含笔者针对思考题查阅的资料和我尝试给出的答案。
- 补充：这里的内容是没有在讲义中提及的问题，以及讲义中建议自学的东西。

Installing GNU/Linux

补充

在安装系统时遇到问题：

> Executing grub_install failed. This is a fatal error.

重启后直接进入 GNU/grub 命令行，说明系统引导出现了问题。经过排查发现使用了与硬盘不匹配的引导方式。

硬盘分区一般有两种格式：MBR 和 GPT。前者是传统的模式，后者较新。笔者因为在一套老式电脑中安装系统所以遇到了这种格式的硬盘。

MBR+LegacyBIOS

MBR分区格式下最多支持四个主分区。如果想划分更多的分区，可以将其中一个主分区变为扩展分区，然后在扩展分区中建立逻辑分区。逻辑分区一般从 5 开始编号。另外 MBR 分区表对硬盘的容量大小有限制。

MBR （Master Boot Record，主引导记录）存放在第一个扇区（一个扇区512字节）中，这个扇区不属于任何一个分区。这个扇区中包含如下内容：

- Bootloader：446字节，内含引导程序，主要负责识别活动分区并引导系统
- DPT (Partition Table，分区表)：64字节，存储了分区信息
- Magic Number: 2字节，固定为0x55AA，结束标志字。

分区表的64个字节分成4组每组16个字节，分别记录了4个主分区的信息。

Partition Flag (00)	Start CHS (01~03)	Partition Bytes (04)	End CHS (05~07)	Start LBA (08~11)	Size (12~15)
00: 不可引导 80: 可以引导	分区开始的 CHS 值	分区类型 83/linux 82/swap 8e/LVM etc.	分区结束的 CHS 值	相对的起始扇区号	该分区拥有的扇区数量

CHS 和 LBA 是两种不同的硬盘寻址方式。

- CHS寻址模式将硬盘划分为磁头 (Heads)，柱面 (Cylinder) 和扇区 (Sector) 。
  - 磁头：每张磁盘的正反面各有一个磁头，一个磁头对应了一个磁面。
  - 柱面：所有磁片中半径相同的同心圆磁道构成柱面。
  - 扇区：将圆环形磁道划分成若干个区段，就是扇区。每个扇区的容量为 512 字节。

  CHS 部分共使用了 3 个字节 24 个二进制位，因此寻址范围是 $2^{24}bit$ ，约为 8G。这种方式较为古老现在已经几乎不用。
- LBA (Logic Block Addressing) 模式中地址不再表示实际的物理地址，而是将 CHS 地址转换成一种唯一的线性地址。由于 LBA 部分共有 32 个二进制位，因此寻址范围扩大到约 2.2TB。这就是前文所说的 MBR 硬盘分区的容量限制，对于更大的硬盘，LBA寻址方式并不能表示其地址。

MBR分区格式的硬盘使用 Legacy BIOS 来引导操作系统。安装系统时应当在 BIOS 中调整引导方式为 `Legacy only` 或将 Legacy 模式的优先级放在 UEFI之前。

以 Windows 系统为例，传统 Legacy 引导操作系统时，会通过活动分区（MBR只允许有一个活动分区）下的 bootmgr（启动管理器）文件导入根目录下 boot 文件夹内的 BCD 文件，然后 BCD 文件根据自身的配置内容加载系统启动文件 `根目录\Windows\system32\winload.exe` 来启动系统。

Linux 使用的引导程序是 GRUB。

GPT+UEFI

GPT分区格式没有4个主分区的限制，且没有主分区、扩展分区、逻辑分区这些概念的区分，对于硬盘的容量也（几乎）没有限制。与GPT对应的UEFI启动方式相较于LegacyBIOS，省去了加电自检的环节，可以更快地引导系统启动，且界面支持鼠标甚至蓝牙等。这都是GPT+UEFI相较于传统的方式优秀的地方。

GPT格式的硬盘有如下部分：

- PMBR (Protective MBR)：占据第一个扇区，其目的是防止只能识别MBR格式的工具误识别并写入硬盘。
- Primary GPT Header：占据第二个扇区
- Partition Entries：占据第三、第四个扇区
- Partitions
- Backup Partition Entries/Primary GPT Header：对之前的信息进行备份。如果表头遭到了误删等操作可以利用备份进行恢复，增加了安全性。

UEFI引导需要硬盘中有一个 EFI 分区。EFI 分区中存放了引导程序，如 Windows 的 bootmgfw.efi，Linux 的 grubx64.efi 等。

装机注意事项

为了同时兼容 MBR 和 GPT 型的硬盘，现在的 BIOS 同时提供 Legacy 和 UEFI 两种启动方式，因此需要根据硬盘将 BIOS/UEFI 调整到正确的模式。对于 MBR 型的硬盘，不需要创建 EFI 分区（安装 Linux 的时候会警告你没有创建 EFI，但没关系），最后的启动分区默认即可。对于 GPT 型的硬盘，可以为新系统创建单独的 EFI 分区，也可以与老系统共用 EFI 分区，但得保证有 EFI 分区。如果启动分区选择了 Linux 的 EFI 分区，将由 Linux 引导 Windows，在 GNU/GRUB 界面上可以选择 Ubuntu 或者 Windows Boot Manager。

First Exploration with GNU/Linux

思考题

> **Why executing the `poweroff` command requires superuser privilege?**
>
> Can you provide a scene where bad things will happen if the `poweroff` command does not require superuser privilege?

例如我有一台服务器，我作为服务器的管理者可以用root的权限对网站进行修改操作。网站的使用者注册账号之后可以用普通账户来访问允许范围内的资源。如果普通账户拥有过大的权限以至于可以修改服务器的系统文件甚至直接关机，那么网站将直接崩溃。

Installing Tools

补充

在安装软件包的过程中遇到依赖问题无法安装，这是因为当前需要安装的软件包依赖其他软件包，而其他软件包在当前的源上找不到。我们可以添加更多的 Ubuntu 源，从而提供其他软件包的资源。

例如使用清华大学的源，在 `/etc/apt/sources.list` 中添加如下语句：

```
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ hirsute main restricted
universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ hirsute-updates main
restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ hirsute-backports main
restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ hirsute-security main
restricted universe multiverse
```

执行 `sudo apt-get update` 之后再下载软件包，即可正常进行。

Configuring Vim

Vim 的 Recording 功能

Recording 功能可以帮助我们完成重复工作。以讲义中的示例为例，欲输出

```
1
2
...
100
```

只需依次按下 `i1<ESC>q1yyp<C-a>q98@1` 即可。

按 `q` 再按下任意一个字符之后进入 Recording 模式。这里的字符相当于一个缓冲器名，表示接下来录制的这段动作将保存在这个缓冲器中，我们可以在不同的缓冲器中录制不同的动作。在示例中，动作被存入了缓冲器 `1` 中。

接下来 `yy` 命令是复制光标所在行整行，`p` 命令是将剪贴板内容粘贴到当前光标下一行并且将光标移动到下一行。`<C-a>/<C-x>` 操作可以进行简单的算数运算，对整数进行+1/-1。

再按一次 `q` 结束当前录制。想要使用之前的动作序列时，先直接按下需要执行的次数，然后按下 `@`，再按下指定的缓冲器名即可。本例中我们直接输入了1，在录制宏的过程中完成了2，因此只需要再执行98次。

Vim 的 Visual Block 模式

Visual Block 模式可以帮助我们对文本进行批量操作。以讲义中的示例为例，欲将下面片段每行左右对调

```
aaaaaaaaaaaaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbbbbbbb
cccccccccccccccccccccccccdddddddddddddddddddddddddd
eeeeeeeeeeeeeeeeeeeeeeeeeffffffffffffffffffffffffff
gggggggggggggggggggggggggghhhhhhhhhhhhhhhhhhhhhhhhh
iiiiiiiiiiiiiiiiiiiiiiiijjjjjjjjjjjjjjjjjjjjjjjjjj
```

只需依次输入 `gg<C-v>24l4jd$p` 即可。

`gg` 命令可以将光标移动到文本开头。按下 `<C-v>` 进入 Visual Block 模式。 `24l4j` 将光标向右移动24格之后再向下移动4格，从而选中了左边一整块所有字符。`d` 操作删除选中的部分，并将删除的内容放入剪贴板。`$` 命令将光标移动到行末（不改变当前的 normal 模式）,`p` 粘贴，从而完成任务。

GNU Diff Format

可以参照 这个文档 来详细了解 GNU diff format。

注意：这种格式只是一种严谨规范地向他人描述修改内容的方法，并不是要将这些内容直接写入要修改的文件！

补充

若使用讲义中的方法直接将 `/etc/vim/vimrc` 复制到 `~/.vimrc`，我们在编辑保存的时候会遇到一些权限问题。

使用 `ls -l -a | grep vimrc` 命令查看 `~/.vimrc` 的详细信息，可以看到其权限信息为 `-rw-r--r--`，这说明除了创建者 `root` 之外别的用户对该文件没有写入的权限。我们可以用 `sudo chmod 664 ~/.vimrc` 命令将其权限修改为 `-rw-rw-r--` 来解决问题。

More Exploration

思考题

> **消失的cd**
>
> 为什么上述基本命令除了 `cd` 之外都能找到它们的 manpage?

`cd` 并不是一条指令，如果在终端中输入 `type cd` 我们会得到 “cd is a shell builtin”。

> **Things Behind Scrolling**
>
> Why the original terminal cannot be scrolled? How does `tmux` make the terminals scrollable? How to implement a scroll bar?

留坑。盲猜最基本的滚轮功能是滚一格向上/下移动 x 行，欲实现滚轮需要将终端内的历史命令和输出保存下来。

补充

正则表达式的最基本语法

`[]` 语法

正则表达式	含义及其说明
`oo`	oo
`[abcd]oo`	aoo/boo/coo/doo `[]` 表示匹配一个字符
`[^ef]oo`	xoo，x不为e和f `^` 是取反符号，且取反中括号内所有列出的字符。

行首行末匹配 `^` `$`

正则表达式	含义及其说明
`^[^A-Z]`	首字符不是大写字母的行 注意 `^` 在中括号中表示取反，不在中括号中表示行首
`\.$`	以 `.` 结尾的行 注意 `.` 有特殊含义，要用 `\` 转义。
`^$`	空白行

任意字符 `.`，重复字符 `*`

正则表达式	含义及其说明
`g..d`	长度为4的以g开头d结尾的串 `.` 表示“一定要有一个字符”
`ooo*`	两个以及更多的o `*` 表示将之前的一个字符重复零次或若干次
`go.*g`	以go开头g结尾的单词，中间可以有任意的字符（也可以为空）
`[0-9][0-9]*`	任意数字

限定次数 `{}`

正则表达式	含义及其说明
`o\{2\}`	o连续出现两次，即oo 注意使用 `{}` 一定要转义
`o\{2,5\}`	o出现两次到五次之间，即oo/ooo/oooo/ooooo
`go\{2,\}d`	开头是g结尾是d,中间出现了两次及以上的o

扩展正则表达式语法

正则表达式	含义及其说明
`go+d`	开头是g结尾是d，中间出现了一次及以上的o `+` 表示前一个字符出现了若干次（与 `*` 的区别是非零）
`go?d`	gd或god `?` 表示前一个字符重复零次或一次
`gd|god`	gd或god `|` 表示或
`g(la|oo)d`	glad/good `()` 里可以罗列若干个串并用 `|` 分隔开
`A(xyz)+B`	开头是A结尾是B中间有一个或多个xyz `()+` 表示括号中的群组重复一次或多次

Standard Streams

> In Computer Programming, **standard streams** are interconnected input and ouput communication channel between a computer program and its environment when it begins execution. (From Wikipedia)

标准流包括三个：标准输入 (stdin, 0)，标准输出 (stdout, 1) 和标准错误 (stderr, 2)。默认情况下，标准输入是通过键盘输入，标准输出和标准错误都是直接打印在终端上通过屏幕显示。可以通过重定向 (redirection) 或管道 （pipeline）来修改标准流。此外，子进程继承父进程的标准流。

关于 `find` 和 `xargs` 命令

在 Linux 入门教程中提到使用 `find . | grep '\.c$|\.h$' | xargs wc -l` 来统计代码的行数。笔者在本地直接使用该命令时报错：`xargs: unmatched single quote`。查阅资料得知这是因为文件名中有奇数个单引号。与此同时，文件名中的空格、斜杠等特殊字符也会导致识别错误。

配合使用 `find` 命令的 `-print0` 参数和 `xargs` 命令的 `-0` 参数可以解决这个问题。它们的作用是用一个 null character 来代替空格分隔文件名，同时可以解决文件名中的空格，斜杠等问题。但我们发现 `find . -print0` 的结果送给 `grep` 无法正常地完成筛选。

我们发现 `find` 命令自带文件名筛选功能。我们可以使用 `-name '*.cpp'` 这个选项来筛选出所有 .cpp 文件，这样使用命令

```
find . -name '*.cpp' -print0 | xargs -0 wc -l
```

可以完成所有 .cpp 文件的代码行数统计，要注意 `-name` 选项后的文件名并不支持完整的正则表达式语法，根据手册，它只支持 `*` `?` 和 `[]` 语法。

如果执意想使用 `grep` 命令的话，我们可以换一个思路来解决这个问题：给文件名加上双引号来解决特殊字符。

```
find . -printf '"%p"\n' | grep '\.cpp"$' | xargs wc -l
```

`-printf` 参数后跟 `%p` 可以输出文件名，我们将所有的文件名用双引号括起来，加一个换行符，就可以让 `find` 在标准输出中将文件名带引号地分行输出，注意此时 `grep` 指令需要匹配行尾的 `"` 。

`/dev/null`，`/dev/zero` 和 `/dev/random`

`/dev/null` 是一个只写文件。如果使用 `cat /dev/null` 查看文件内容会得到空。通常可以通过将标准输出重定向到 `/dev/null` 的方法来吃掉你不想要的输出内容。虽然无法写入，但在写入时它会返回给你写入成功。

`/dev/zero` 在读取时会提供无限的空字符。可以用来覆盖文件内容或产生特定大小的空文件。

`dev/random` 在读取时会产生永不为空的随机字节数据流。

对于这些特殊文件实现方式的一个简单解释是，例如 `/dev/null` ,当内核接收到对这个文件的请求时，不需要进行任何真实的硬件操作，直接按照API返回写入成功即可。

Makefile

Makefile最简单的一点语法：

```
要生成的文件名:依赖文件列表
	用于生成目标文件的命令序列
```

如果连续多次执行 `make` ，会得到信息 'xxx is up to date'。这是 `make` 程序智能管理的功能，如果目标文件已经存在且修改时间后于其依赖的所有文件，`make` 会认为这个文件已经编译过了，不需要重新编译。

（即使这个文件并不是按照命令要求生成的，如在当前目录下有文件 `hello.c` 和 `Makefile`。`Makefile` 的内容如下：

```
hello:hello.c
    gcc hello.c -o hello
```

此时用 `touch hello` 指令创建一个修改时间最新的 `hello` 空文件再执行 `make` ，仍然显示 'hello is up to date'。）

使用语句 `:PHONY xxx` 可以将 `xxx` 变成一个伪目标，这样以后在执行 `make xxx` 时不会再检查文件 `xxx` 的新旧，而是直接执行 `xxx` 规则下的命令。

Getting Source Code for PAs

思考题

> **What happened?**
>
> You should know how a program is generated in the 程序设计基础 course. But do you have any idea about what happened when a bunch of information is output to the screen during `make` is executed?

留坑。

补充

Git使用 - 关于 SSH Key 的设置

如果想要向远程仓库提交修改，必须将本机的 SSH Key 添加到受信任的 SSH Key 列表中。

如果本地没有 SSH Key，可以使用命令 `ssh-keygen -t rsa -C "email"` 命令来生成 SSH Key，其中 email 是在 `git config --global user.email` 中填写的邮箱。默认生成位置在 `~/.ssh` 中。生成后将公钥 `~/.ssh/id_rsa.pub` 中的内容填写到受信任 SSH Key 列表中即可。

关于 `make menuconfig` 的报错

在 `ics2021/nemu` 下使用命令：`make menuconfig`，报错 `bison: No such file or directory` 。

报错是因为缺少 bison 工具，安装即可。对于之后的 flex 工具也是亦然。

注：当你缺少某项工具时，不一定会返回 `command not found`，也可能会返回 `No such file or directory`！

关于软件安装的报错

在使用 `apt-get install` 安装软件时若出现形如以下的错误：

```
E: Failed to fetch
http://mirrors.tuna.tsinghua.edu.cn/ubuntu/pool/main/f/flex/flex_2.6.4-
8_amd64.deb  Connection timed out [IP: 2402:f000:1:400::2 80]
E: Unable to fetch some archives, maybe run apt-get update or try with --fix-
missing?
```

虽然可能是源中没有对应的 package，但 `Connection timed out` 也可能是因为网速不行，重新安装一次也许就可以解决。

PA1 - 开天辟地的篇章：最简单的计算机

实验进度

我完成了所有的必答题，并针对选做题和思考题给出了自己的想法。

必答题

程序是个状态机

点击 这里 跳转到解答。

理解基础设施

有 450 次编译用于调试，假设只有 450 个 bug，那么总共需要 4500min = 75h 。简易调试器可以节约 50h 的时间。

RTFM

> riscv32有哪几种指令格式?

在 Volume I - 2.2 节可以查到 Base Instruction Formats, 在 Volume I -17.2 节中可以查到 Compressed Instruction Formats。在 Volume I - 26.3 节可以查到扩展 RISCV 的指令格式。

> LUI指令的行为是什么?

在 Volume I 第 19 页可以查到 LUI 指令的行为。

> mstatus寄存器的结构是怎么样的?

在 Volume II - 3.1.6 节可以查到 machine status register 的结构。

Shell命令

笔者使用了如下命令来统计所有非空行的代码行数：

```
find . -name "*[.c|.h]" | xargs grep ^. | wc -l
```

其中的正则表达式 `^.` 用来识别开头至少有一个字符的行。

pa0结束时共有代码 19868 行，pa1结束时共有代码 20272 行。

RTFM

`-Wall` 选项会显示所有的 warning 信息，`-Werror` 会将所有 warning 视作 error 报错。使用这些编译选项可以尽早发现潜在的错误并强制报错，将 bug 消灭在 fault 的阶段。

以下是我的自由报告内容，我在做实验的过程中详细记录了自己遇到的所有问题和查阅的所有资料。这些笔记按照章节排列，每章通常有两个部分：

- 思考题：包含笔者针对思考题查阅的资料和我尝试给出的答案。
- 补充：这里的内容是没有在讲义中提及的问题，以及讲义中建议自学的东西。

在开始愉快的PA之旅之前

补充

关于 `git status` 追踪不到修改

在 `ics2021` 根目录下新建文件 `touch test.cpp` 后使用 `git status` 查看变化，发现 git 并没有追踪到这条变化。使用 `git add a.cpp` 命令后显示：

```
The following paths are ignored by one of your .gitignore files
```

这说明根目录的 `.gitignore` 文件忽略了该文件，打开 `.gitignore` 查看：

```
*.*
*
!*/
!/nemu/*
!/nexus-am/*
!/nanos-lite/*
!/navy-apps/*
!Makefile
!README.md
!.gitignore
!init.sh
/fceux-am
/am-kernels
```

查阅资料得知，`.gitignore` 文件的最基础语法包括：

- 可以用正则表达式 `[]` `?` `*` 等来匹配文件名
- 行开头使用 `!` 表示不忽略该行匹配的文件

`ics2001/a.cpp` 被 `*.*` 捕捉，因此被 git 忽略了。为了验证我的思考，我在 `ics2001/nemu` 下创建了 `a.cpp` ，根据根目录下的 `.gitignore` 文件 `/nemu` 下的文件没有被忽略，但 git 仍然没有追踪到 `a.cpp`。

我发现 `/nemu` 下也有 `.gitignore` 文件，该文件中的 `*.*` 捕捉了 `a.cpp`。查阅资料得知，当多个 `.gitignore` 文件发生冲突时，git 优先服从本地的文件而不是父文件夹的文件。

关于添加环境变量

`ccache` 工具要求将 `/usr/lib/ccache` 添加到 PATH 中。笔者一开始直接在 `~/.bashrc` 中添加了如下一行：

```
export PATH=/usr/lib/ccache
```

之后发现其他所有的命令都用不了了。事实上大部分命令都放在 `/usr/bin` 之下，这个路径默认保存在 PATH 中。如果向上文那样直接修改 PATH，会导致其他命令无法使用。

在命令行中输入 `export PATH="/usr/bin:$PATH"` 可以暂时将 `/usr/bin` 添加回 PATH，但终端关闭后会失效。此时打开 `~/.bashrc` 将命令修改为

```
export PATH=/usr/lib/ccache:$PATH
```

后一切正常。这里的 `:` 是多个路径之间的分隔符，这条语句相当于在原本的 `$PATH` 内容之前加上 `/usr/lib/ccache`。

（事实上仔细阅读 `ccache` 的文档可以发现它让你将路径 “perpend” 到 PATH 中，没让你直接修改……）

开天辟地的篇章

必答题

从状态机视角理解程序运行

> 以上一小节中`1+2+...+100`的指令序列为例, 尝试画出这个程序的状态机.
>
> 这个程序比较简单, 需要更新的状态只包括`PC`和`r1`, `r2`这两个寄存器, 因此我们用一个三元组`(PC, r1, r2)`就可以表示程序的所有状态, 而无需画出内存的具体状态. 初始状态是`(0, x, x)`, 此处的`x`表示未初始化. 程序`PC=0`处的指令是`mov r1, 0`, 执行完之后`PC`会指向下一条指令, 因此下一个状态是`(1, 0, x)`. 如此类推, 我们可以画出执行前3条指令的状态转移过程:
>
> ```
> (0, x, x) -> (1, 0, x) -> (2, 0, 0) -> (3, 0, 1)
> ```
>
> 请你尝试继续画出这个状态机, 其中程序中的循环只需要画出前两次循环和最后两次循环即可.

完整的状态机如下：

```
(0,x,x) -> (1,0,x) -> (2,0,0) -> (3,0,1) -> (4,1,1) -> (2,1,1) -> (3,1,2) ->
(4,3,2) -> (2,3,2) -> ... -> (2,4851,98) -> (3,4851,99) -> (4,4950,99) ->
(2,4950,99) -> (3,4950,100) -> (4,5050,100) -> (5,5050,100) -> (5,5050,100) ->
...
```

思考题

[二周目] 计算机可以没有寄存器吗？

> 如果没有寄存器, 计算机还可以工作吗? 如果可以, 这会对硬件提供的编程模型有什么影响呢?
>
> 就算你是二周目来思考这个问题, 你也有可能是第一次听到"编程模型"这个概念. 不过如果一周目的时候你已经仔细地阅读过ISA手册, 你会记得确实有这么个概念. 所以, 如果想知道什么是编程模型, RTFM吧.

留坑，二周目再做。

RTFSC

思考题

kconfig生成的宏与条件编译

> 我们已经在上文提到过, kconfig会根据配置选项的结果在`nemu/include/generated/autoconf.h`中定义一些形如`CONFIG_xxx`的宏, 我们可以在C代码中通过条件编译的功能对这些宏进行测试, 来判断是否编译某些代码. 例如, 当`CONFIG_DEBUG`这个宏没有定义时, 调试相关的代码就无需进行编译.
>
> 为了编写更紧凑的代码, 我们在`nemu/include/debug.h`中定义了一些专门用来对宏进行测试的宏. 例如`IFDEF(CONFIG_DEVICE, init_device());`表示, 如果定义了`CONFIG_DEVICE`, 才会调用`init_device()`函数; 而`MUXDEF(CONFIG_DEBUG, "ON", "OFF")`则表示, 如果定义了`CONFIG_DEBUG`, 则预处理结果为`"ON"`(`"OFF"`在预处理后会消失), 否则预处理结果为`"OFF"`.
>
> 这些宏的功能非常神奇, 你知道这些宏是如何工作的吗?

宏定义的基本格式为 `#define A B`，这里的 `A` 可以包含参数也可以不包含。在编译的过程中，程序中的 A 会被替换成 B。

条件编译指类似于 `#idef A` `#ifndef A` 等，表示如果宏 `A` 有定义/无定义才会编译下面的代码，以 `#endif` 结尾。

为什么全部都是函数?

> 阅读`init_monitor()`函数的代码, 你会发现里面全部都是函数调用. 按道理, 把相应的函数体在`init_monitor()`中展开也不影响代码的正确性. 相比之下, 在这里使用函数有什么好处呢?

事实上，如果能确定这些代码块只会使用一次，那么将这些代码放进函数体，并在每块的开头用注释写上该部分的作用，也可以达到代码易阅读的效果；但阅读代码发现这些代码块在其他的函数体中也有使用，因此封装成函数可以减少代码量。从另一种角度，这样写 `init_monitor()` 函数内容更加清爽，可以更好地体现分层抽象。

究竟要执行多久？

> 在`cmd_c()`函数中, 调用`cpu_exec()`的时候传入了参数`-1`, 你知道这是什么意思吗?

`cpu_exec()` 中的参数 `n` 代表的是 cpu 执行指令的次数。在 `/nemu/src/cpu/cpu_exec.c` 的源码中是这样写的

```
for (;n>0;n--) {
    ...
}
```

因此传入参数 `-1` 可以使 cpu 不停地执行指令，直到遇到 nemu_trap 指令。

参数的处理过程

> 另外的一个问题是, 这些参数是从哪里来的呢?

这些参数来自命令行，是用户键入的。main() 函数自带的两个参数 `int argc` 和 `char *argv[]` 处理用户在命令行中键入的参数。`argc` 是参数的个数，`char *argv[]` 是指向参数的指针数组。`parse_args()` 中传递的参数就是这两个。

[二周目] 潜在的威胁

> "调用`cpu_exec()`的时候传入了参数`-1`", 这一做法属于未定义行为吗? 请查阅C99手册确认你的想法.

留坑，二周目再做。

谁来指示程序的结束？

> 在程序设计课上老师告诉你, 当程序执行到`main()`函数返回处的时候, 程序就退出了, 你对此深信不疑. 但你是否怀疑过, 凭什么程序执行到`main()`函数的返回处就结束了? 如果有人告诉你, 程序设计课上老师的说法是错的, 你有办法来证明/反驳吗? 如果你对此感兴趣, 请在互联网上搜索相关内容.

main() 函数是程序的入口，但 main () 函数也是被其他程序调用的。例如有一个可执行文件 `test`，在 shell 中输入 `./test` 之后，shell 会 clone 一个子进程，执行

```
execve("./test",char * const argv[], char * const envp[])
```

execve 加载 ./test 并将参数一步步传递下去。从 `./test` 的入口开始执行（ELF文件中的 `_start()`），`_start()` 调用 `__libc_start_main()`。`__libc_start_main()` 中有一条

```
int result = main (argc, argv, __environ MAIN_AUXVEC_PARAM);
```

从这里进入 main() 函数，main() 函数的返回值会存在 result 中，`__libc_start_main()` 会直接 `exit(result)` ，从而 main() 函数的返回值会被父进程捕捉到。

执行完 `./test` 后用 `echo $>` 指令可以打印 main() 函数的返回值。

[二周目] 有始有终

> 对于GNU/Linux上的一个程序, 怎么样才算开始? 怎么样才算是结束? 对于在NEMU中运行的程序, 问题的答案又是什么呢?
>
> 与此相关的问题还有: NEMU中为什么要有`nemu_trap`? 为什么要有monitor?

留坑，二周目再做。

补充

关于 `$@` 和 `$<`

`$@` 代表目标文件，`$<` 代表第一个依赖文件，此外类似的字符还有 `$^` 代表所有的依赖文件。

关于 `getopt` 和 `getopt_long`

两者都是用于分析参数，不同的是 `getopt` 会忽略长参数（即由 `--` 开头的参数，参数名字可以不止一个字符）而 `getopt_long` 不会。

`monitor.c` 中的源码如下

```
const struct option table[] = {    {"batch"    , no_argument       , NULL, 'b'},
   {"log"      , required_argument, NULL, 'l'},    {"diff"     ,
required_argument, NULL, 'd'},    {"port"     , required_argument, NULL, 'p'},
   {"help"     , no_argument       , NULL, 'h'},    {0          , 0
   , NULL,  0 },};int o;while ( (o = getopt_long(argc, argv, "-bhl:d:p:",
table, NULL)) != -1) {    switch (o) {      case 'b': sdb_set_batch_mode();
break;      case 'p': sscanf(optarg, "%d", &difftest_port); break;      case
'l': log_file = optarg; break;      case 'd': diff_so_file = optarg; break;
 case 1: img_file = optarg; return optind - 1;      default:
 printf("Usage: %s [OPTION...] IMAGE [args]\n\n", argv[0]);        printf("\t-
b,--batch              run with batch mode\n");        printf("\t-l,--log=FILE
          output log to FILE\n");        printf("\t-d,--diff=REF_SO        run
DiffTest with reference REF_SO\n");        printf("\t-p,--port=PORT
run DiffTest with port PORT\n");        printf("\n");        exit(0);    }
```

- `getopt()` 里有如下参数：`int argc` 表示要分析的选项个数，`char *argv[]` 是指向参数的选项数组。`getopt()` 有一个额外的变量 `optind` 记录当前已经分析到第几个选项。每调用一次，`getopt()` 会返回下一个选项剥去了 `-` 的结果 (optional character)，如果已经没有下一个选项则返回 `-1`。
- `getopt()` 的参数 `optstring` 表示识别哪些选项。如果选项字符前有 `:` 的话表示该选项还有参数，`getopt()` 分析时会将该选项的参数保存在 `optarg` 变量中。

  `getopt()` 在默认情况下会将 `argv` 数组中的选项重排，将那些不是选项的 args 放到数组的最后。有两种特殊的扫描方式可以无视这种默认：

  - `optstring` 的首字母是 `+`，表示遇到第一个非选项的 args 就退出
  - `optstring` 的首字母是 `-`，表示处理所有的 args，如果遇到非选项的 args，返回的 optional character 是 `1`。可以看到代码中对于非选项的部分 (Case 1) 返回了 `optind-1`，转到异常处理。
- `switch` 语句中没有判断 `h`，因为 `h` 是 default 的情况。

基础设施

思考题

如何测试字符串处理函数？

> 你可能会抑制不住编码的冲动: 与其RTFM, 还不如自己写. 如果真是这样, 你可以考虑一下, 你会如何测试自己编写的字符串处理函数?
>
> 如果你愿意RTFM, 也不妨思考一下这个问题, 因为你会在PA2中遇到类似的问题.

笔者的一点思考是，在测试的时候一定要仔细考虑字符串行末的 '\0' 问题。笔者在实现简易的表达式求值函数时，在 `make_token` 函数中使用 C语言自带的 `strncpy` 函数来将一个字符串的一部分拷贝进 tokens 数组中，但在测试时发现如果 `p` 过之后 `x` 再 `p` 会打印意想不到的结果。

仔细阅读 `strncpy` 的手册，发现其中有这样一句话：

```
Warning: If there is no null byte among the first n bytes of src, the string
placed in dest will not be null-terminated.
```

这正好是笔者在复制 tokens 过程中遇到的情况。因为复制过来的字符串不带结束符，所以 strlen 函数不能准确给出字符串长度，从而出现错误。

补充

RTFSC 技巧

如果遇到一个宏或者一个函数不知道该到哪里去找来源，可以使用 `grep xxx -r` 命令来寻找所有包含关键字 `xxx` 的文件。当然在使用了现代的IDE（如 vscode）后这方面的焦虑会少很多。

关于在 nemu 中直接用 `q` 退出会报错 `Error 1`

在 nemu 的主程序 `/nemu/src/nemu-main.c` 中最后调用了函数 `is_exit_status_bad()`，该函数在 `/nemu/src/utils/state.c` 中，源码如下：

```
int is_exit_status_bad() {  int good = (nemu_state.state == NEMU_END &&
nemu_state.halt_ret == 0) ||     (nemu_state.state == NEMU_QUIT);  return
!good;}
```

可以看到正常退出的两个条件

- 程序运行结束且没有遇到任何非正常情况
- 执行了 quit 操作

判断执行 quit 操作的方法是 `nemu_state.state == NEMU_QUIT`，然而在框架代码中的 `cmd_q()` 中只有一行 `return -1`。这里的 `return -1` 只是用于给 `sdb_mainloop()` 传递信息使其结束，但没有做好标志信息的设置。所以修改 `cmd_q()` 为

```
static int cmd_q()
{
    nemu_state.state = NEMU_QUIT;
    return -1;
}
```

再次编译后，即使在程序未运行结束时 quit 也不会收到报错。

关于在 `sdb.c` 中使用访问内存函数报错

在扫描内存函数中，笔者使用 `paddr.c` 中定义的函数 `guest_to_host()` 函数来访问内存，但运行后显示

```
error: implicit declaration of function 'guest_to_host'
```

这是因为程序找不到该函数的定义，我们可以 `#include <memory/paddr.c>` 来解决这个问题。

内置客户代码放在了哪里？

跟着程序运行的过程 RTFSC 是一个不错的方法，不过更简单的是我们可以充分相信自己写的程序的正确性：直接在 `nemu` 文件夹内搜索打印出的内存信息。迅速定位到在 `/nemu/src/isa/init.c` 中的代码：

```
0x800002b7,  // lui t0,0x80000
0x0002a023,  // sw  zero,0(t0)
0x0002a503,  // lw  a0,0(t0)
0x0000006b,  // nemu_trap
```

和打印得到的信息比对，我们可以看到 riscv 是一个小端机器。在 `init_isa` 函数中是通过

```
memcpy(gueset_to_host(RESET_VECTOR), img, sizeof(img))
```

的方式将 img 中的代码拷贝进内存的。另外我们可以看到：内存中未初始化的部分是乱码。

表达式求值

思考题

为什么printf()的输出要换行?

> 如果不换行, 可能会发生什么? 你可以在代码中尝试一下, 并思考原因, 然后STFW对比你的想法.

留坑。

[二周目] 为什么要使用无符号类型?

> 我们在表达式求值中约定, 所有运算都是无符号运算. 你知道为什么要这样约定吗? 如果进行有符号运算, 有可能会发生什么问题?

留坑。一个想法是引入负数可能会在访存时发生意想不到的问题，但这个问题似乎可以通过访存前的判断来筛除。

表达式生成器如何获得C程序的打印结果?

> 代码中这部分的内容没有任何注释, 聪明的你也许马上就反应过来: 竟然是个RTFM的圈套! 阅读手册了解API的具体行为可是程序员的基本功. 如果觉得去年一整年的程序员都白当了, 就从现在开始好好锻炼吧.

`gen-expr.c` 中使用了如下几个值得借鉴的函数：

- `sprintf(code_buf, code_format, buf)` ，该函数用于将 buf 的内容填入到 code_format 的 "%s" 中，并将整段c语言代码的字符串放进 code_buf。
- `system()`，用于执行一个命令，返回值是为该命令创建的子进程的返回值，如果不是 0 说明指令执行失败。
- `fopen/popen ("file", "r"/"w")` 用于打开文件，后面的 `r/w` 指明是读取还是写入。根据 STFW 的结果，两者的区别在于 `popen()` 会 fork 一个子进程并建立管道连接（不懂，留坑）。经测试，第一处 `fopen` 如果换成 `popen` 会在写入时报错 `Permission denied`；第二处 `popen` 如果换成 `fopen` 则无法读到程序的输出结果。

补充

关于表达式生成

1. 如果直接将生成的表达式喂给C语言，C语言会把常量当做整型参与运算，从而得到不正确的结果，如在下面的例子中

   ```
   (0 - 1) / 10
   ```

   在 int 规则下结果为 0，赋给 unsigned 变量之后结果仍是0，而在 unsigned 规则下结果显然不为 0。unsigned 可以看做 int 的一个值域右移的版本，在这个版本下做任何加减操作不会影响正确性，但做乘除操作时会发生问题。

   笔者的解决方案是：生成两个表达式，一个正常的表达式用于喂给 nemu，另一个表达式在每个常量后面添加一个 `u`，将常量转化为 unsigned 类型后再喂给C语言。

2. 笔者并不希望通过计算的方式来过滤除0的表达式，因此使用了一种比较无脑的暴力方法：

笔者修改了生成表达式的 BNF 规范，在新规范中没有 `<expr> / <expr>` ，而是 `<expr> / ((<expr>)*2+ 1)` 。无符号数可以看作做完所有操作以后对 2^{32} 取模，由于模数是偶数，因此 `<expr>*2+1` 取模过后必然是一个奇数，这样可以保证除数非零。这样做的唯一缺陷是生成的除数不再是等概率分布的，不过在测试正确性的过程中除数分布的均匀无关紧要。

3. 空格的插入是容易的，只需要在 BNF 规范中添加一条：`<expr> ::= " "<expr>` 即可。
4. 在当前的随机生成方式下，生成长表达式的概率较小。如果想生成更多的表达式，可以修改随机到各个部分的概率，使 `<expr> ::=  <decimal number> | <heximal number>` 的概率变小。或者设置一个下界，当表达式总长小于下界时不允许生成 `<number>`。

   控制表达式的长度不超过 buffer 的限制是容易的：我们在生成表达式的同时记录当前表达式的总长，当总长超过一个预警值时，强制新表达式只能生成常数即可。

关于负数功能

笔者在 PA1 中实现了负数功能。负号的识别和指针解引用是类似的：如果一个 `-` 的前面是双目运算符，负号或者 `(` 的话，那么当前的 `-` 就是负号（单目运算符）。

在计算的过程中，如果一个表达式由负号开头，且后面的部分中没有“裸露在外”（即不被任何一对括号包裹）的双目运算符，则可以剥离这个负号递归计算，将得到的答案取负之后返回。

监视点

思考题

温故而知新

> 框架代码中定义`wp_pool`等变量的时候使用了关键字`static`, `static`在此处的含义是什么? 为什么要在此处使用它?

static 最重要的作用在于隐藏。带有 static 的变量以及函数不能在该文件以外的地方访问和使用。例如在 `cpu-exec.c` 文件中，我们要实现对所有监视点的扫描，但又不能直接访问 `watchpoint.c` 中的监视点池，因此我们可以在 `watchpoint.c` 中写一个函数实现这个功能，再通过在 `cpu-exec.c` 中调用这个非 static 的函数来完成任务。使用 static 可以保证本文件中的东西不会被外界污染，如果出现问题，可以在本文件中寻找 bug，降低了调试难度。

你会如何测试你的监视点实现?

> 我们没有提供监视点相关的测试, 思考一下, 你会如何测试?
>
> 当然, 对于实验来说, 将来边用边测也是一种说得过去的方法, 就看你对自己代码的信心了.

笔者没有批量生成测试用例，而是针对监视点的各个功能设计了几种测试手段：

- 每执行完一次操作都使用命令 `info w` 检查所有监视点是否正常。
- 使用表达式 `w *0x80000000+1` ，检查指针解引用部分的正确性， 以及监视点遇到变化是否能暂停程序。
- 使用表达式 `w $t0 / 5`，检查寄存器访问部分的正确性（`t0` 是内置客户程序中唯一修改过的寄存器）。
- 检测 `d` 命令是否能实现，以及删除不存在的监视点是否可以报错。
- 尝试添加大于监视点池容量的监视点，检查是否能触发 assertion fail。

强大的GDB

> 如果你遇到了段错误, 你很可能会想知道究竟是哪一行代码触发了段错误. 尝试编写一个触发段错误的程序, 然后在GDB中运行它. 你发现GDB能为你提供哪些有用的信息吗?

笔者尝试在 nemu 中输入指令 `x 10 0x0`。由于 riscv32 的地址从 0x80000000 开始，该命令在计算地址时会得到负数从而引发段错误。GDB 输出了以下信息：

```
Program received signal SIGSEGV, Segmentation fault.0x0000555555562cfd in cmd_x
()
```

GDB 可以定位到引发段错误的函数 `cmd_x()`。

[二周目] 如何提高断点的效率

> 如果你在运行稍大一些的程序(如microbench)的时候使用断点, 你会发现设置断点之后会明显地降低NEMU执行程序的效率. 思考一下这是为什么? 有什么方法解决这个问题吗?

留坑。

一点也不能长?

> x86的`int3`指令不带任何操作数, 操作码为1个字节, 因此指令的长度是1个字节. 这是必须的吗? 假设有一种x86体系结构的变种my-x86, 除了`int3`指令的长度变成了2个字节之外, 其余指令和x86相同. 在my-x86中, 上述文章中的断点机制还可以正常工作吗? 为什么

x86架构支持使用8位的 `int` 指令来实现中断，其中 `int3` 为CPU所有，专门用于 debugger 的中断。它的原理是将 `0xCC` 换到对应指令的第一个字节。这样 CPU 在读取指令时遇到 `0xCC` 就会暂停程序的执行。之所以使用单字的指令，是因为这样不论设置断点的指令是单字、双字还是四字，我们都可以在本条指令内部完成 `int3` 的标记，不会写到下一条指令中。

在本题中，如果使用2个字节的指令，则在对单字指令进行标记时会写到别的存储区域中，从而可能出错。

随心所欲的断点

> 如果把断点设置在指令的非首字节(中间或末尾), 会发生什么? 你可以在GDB中尝试一下, 然后思考并解释其中的缘由.

笔者学习了如何在指定的地址添加断点。对于一个输出 hello world 的简单C程序，用 gdb 运行后设置断点 `b *main`，运行暂停后输入命令 `disassemble`，可以看到

```
=> 0x0000555555555149 <+0>: endbr64     0x000055555555514d <+4>: push   %rbp
0x000055555555514e <+5>:  mov    %rsp,%rbp    0x0000555555555151 <+8>: sub
 $0x10,%rsp    0x0000555555555155 <+12>:   movl   $0x1,-0xc(%rbp)
0x000055555555515c <+19>:  movl   $0x2,-0x8(%rbp)    0x0000555555555163 <+26>:
mov    -0xc(%rbp),%edx    0x0000555555555166 <+29>:  mov    -0x8(%rbp),%eax
0x0000555555555169 <+32>:  add    %edx,%eax    0x000055555555516b <+34>:    mov
   %eax,-0x4(%rbp)    0x000055555555516e <+37>:  mov    -0x4(%rbp),%eax
0x0000555555555171 <+40>:  mov    %eax,%esi    0x0000555555555173 <+42>:    lea
   0xe8a(%rip),%rdi        # 0x555555556004    0x000055555555517a <+49>: mov
$0x0,%eax    0x000055555555517f <+54>:    call   0x555555555050 <printf@plt>
0x0000555555555184 <+59>:  mov    $0x0,%eax    0x0000555555555189 <+64>:
leave     0x000055555555518a <+65>: ret
```

我们可以看到每条汇编指令的地址。笔者设置了断点 `b *main+55` ，该位置不在指令的首字节，继续运行后 GDB 没有检测到这个断点。笔者认为 GDB 检测断点的机制是每次取指译码时根据首字节是否是 0xCC 来判断是否有断点，因此将标志信息设置在非首字节是无法被识别的。

NEMU的前世今生

> 你已经对NEMU的工作方式有所了解了. 事实上在NEMU诞生之前, NEMU曾经有一段时间并不叫NEMU, 而是叫NDB(NJU Debugger), 后来由于某种原因才改名为NEMU. 如果你想知道这一段史前的秘密, 你首先需要了解这样一个问题: 模拟器(Emulator)和调试器(Debugger)有什么不同? 更具体地, 和NEMU相比, GDB到底是如何调试程序的?

留坑。笔者暂时认为调试器是可以接触底层的硬件和数据的，为程序员反馈信息的应用程序，NEMU 相当于内置了一个调试器的功能，其本身不只是应用程序，而是一整套虚拟机。

补充

关于 `sscanf` 函数

笔者之前在处理指令中的常数时是通过一位一位读取并计算的方式来将其转换为 int 型的。在 RTFSC 的过程中发现 `sscanf` 函数可以简洁地实现这个功能。例如有 int 型变量 N，有存储着常数的字符串 arg，则可以通过 `sscanf(arg, "%d", &N)` 来将 arg 的内容转换成整数存进 N。

Address Sanitizer 实现 use-after-free 检测的原理

留坑。

如何阅读手册

必答题

尝试通过目录定位关注的问题

> 假设你现在需要了解一个叫`selector`的概念, 请通过i386手册的目录确定你需要阅读手册中的哪些地方. 即使你选择的ISA并不是x86, 也可以尝试去查阅这个概念.

在 I386 手册的 5.1.3 节中可以找到和 selector 有关的内容。

PA2 - 简单复杂的机器: 冯诺依曼计算机系统

实验进度

我完成了所有的必答题，并针对选做题和思考题给出了自己的想法。

必答题

程序是个状态机

点击这里跳转到解答。

RTFSC

点击这里跳转到解答。

程序如何运行

点击这里跳转到解答。

编译与链接

> 在`nemu/src/engine/interpreter/rtl-basic.h`中, 你会看到由`static inline`开头定义的各种RTL指令函数. 选择其中一个函数, 分别尝试去掉`static`, 去掉`inline`或去掉两者, 然后重新进行编译, 你可能会看到发生错误. 请分别解释为什么这些错误会发生/不发生? 你有办法证明你的想法吗?

如果去掉 inline，编译时会报错：`xxx defined but not used`。这是因为 `rtl.h` 文件 include 了 `rtl-basic.h` ，但并没有使用这个函数，因此触发了 `-Werror` 编译选项。

如果去掉 static，编译时不会报错。这是因为这里的函数被定义成了内联函数，函数内容被直接塞进了调用者的函数体中。如果我们用 `objdump -d` 命令去检查编译得到的汇编程序，我们会发现 `rtl_xx` 等一系列函数是不在其中的。

如果同时去掉 static 和 inline，编译时会报错：`multiple definition of xxx`。此时再查看汇编程序可以发现 `rtl_xx` 在其中。因为 `rtl_xx` 同时出现在了 `cpu-exec.o` `hostcall.o` 和 `decode.o` 中，所以链接时会出错。这里一个值得关注的细节是：我们编写的函数 `exec_addi` 调用了 `rtl_addi` ，但在汇编代码中并没有这一条调用。可以看到 `rtl_addi` 的内容被直接贴进了 `exec_addi` 的函数体中。这应该是编译器针对 addi 这种极其简短的函数调用做出的一种优化。

编译与链接2

> 1. 在`nemu/include/common.h`中添加一行`volatile static int dummy;` 然后重新编译NEMU. 请问重新编译后的NEMU含有多少个`dummy`变量的实体? 你是如何得到这个结果的?
> 2. 添加上题中的代码后, 再在`nemu/include/debug.h`中添加一行`volatile static int dummy;` 然后重新编译NEMU. 请问此时的NEMU含有多少个`dummy`变量的实体? 与上题中`dummy`变量实体数目进行比较, 并解释本题的结果.
> 3. 修改添加的代码, 为两处`dummy`变量进行初始化:`volatile static int dummy = 0;` 然后重新编译NEMU. 你发现了什么问题? 为什么之前没有出现这样的问题? (回答完本题后可以删除添加的代码.)

1. 笔者起初的做法是在 Makefile 中加入一些功能，使得在 make 可以将预编译的 `*.i` 文件同时输出到 `/build` 中，然后在 `/build` 中使用 `grep "volatile static int dummy" -r | wc -l` 命令统计个数，得到的答案是 33 个。

   但这个做法其实有一些问题，因为高级程序中定义的“变量”并不一定是实体，比如连续定义两个相同名字的未初始化全局变量，在符号表中只会出现一个。因此正确的方法应该是查看符号表。使用命令 `readelf --symbols $NEMU_HOME/build/riscv32-nemu-interpreter | grep "dummy" | wc -l` 命令，得到的结果是 33 个。

2. 此时 nemu 中仍然含有 33 个 dummy 变量的实体。这是因为 `common.h` 中 include 了 `dubug.h` ，且这是唯一一个 include `debug.h` 的地方，未初始化的相同名字的全局变量在符号表中只有一个实体。

3. 对变量进行初始化之后再编译，会报错：`redefinition of dummy`。这是因为赋了初值的全局变量会被认为是一个强符号。C 语言中不允许有两个相同的强符号被定义。而没有赋初值的变量是弱符号，语法上是可以重复定义的。

了解 Makefile

> 请描述你在`am-kernels/kernels/hello/`目录下敲入`make ARCH=$ISA-nemu`后, `make`程序如何组织.c和.h文件, 最终生成可执行文件`am-kernels/kernels/hello/build/hello-$ISA-nemu.elf`. (这个问题包括两个方面:`Makefile`的工作方式和编译链接的过程.) 关于`Makefile`工作方式的提示:
>
> - `Makefile`中使用了变量, 包含文件等特性
> - `Makefile`运用并重写了一些implicit rules
> - 在`man make`中搜索`-n`选项, 也许会对你有帮助
> - RTFM

hello 中的 Makefile 内容比较简单：将 NAME 和 SRC 设置好，然后将 AM 中的 Makefile 全部贴进来。（虽然 AM 的 Makefile 相当复杂）

事实上 SRC 有非常多：AM 中几乎每个文件夹下都有 Makefile，把这个文件夹下的 .c 文件搜刮进 SRC 中。

按照正序，Makfile 大概做了如下一些大的事情：

- 在 `Makefile` 中有

  ```
  $(DST_DIR)/%.o: %.c
      @mkdir -p $(dir $@) && echo + CC $<
      @$(CC) -std=gnu11 $(CFLAGS) -c -o $@ $(realpath $<)
  ```

  它会中所有的.c文件编译成.o文件放进目录DST_DIR中。这里DST_DIR是 AM 下的 build 目录。

- 在 `Makefile` 中有

  ```
  $(LIBS): %:
      @$(MAKE) -s -C $(AM_HOME)/$* archive
  ```

  它会把我们自己写的库函数（如 klib）打包成 archive。

- 在 `Makefile` 中有

  ```
  $(IMAGE).elf: $(OBJS) am $(LIBS)
      @echo + LD "->" $(IMAGE_REL).elf
      @$(LD) $(LDFLAGS) -o $(IMAGE).elf --start-group $(LINKAGE) --end-group
  ```

  它会将所有的 .o 文件全部链接起来生成一个 .elf 文件。这里的 `$(OBJS)` 的生成方法不难：将 SRC 中所有的 .c 换成了 .o，加上相对应的路径前缀即可。

- 在 `nemu.mk` 中有

  ```
  image: $(IMAGE).elf
      @$(OBJDUMP) -d $(IMAGE).elf > $(IMAGE).txt
      @echo + OBJCOPY "->" $(IMAGE_REL).bin
      @$(OBJCOPY) -S --set-section-flags .bss=alloc,contents -O binary
  $(IMAGE).elf $(IMAGE).bin
  ```

  它使用 `OBJCOPY` 命令将 .elf 文件中的一些节做了修改，然后粘贴进了 .bin 文件中。

- 在 `nemu.mk` 中有

```
run: image
    $(MAKE) -C $(NEMU_HOME) ISA=$(ISA) run ARGS="$(NEMUFLAGS)"
IMG=$(IMAGE).bin
```

它相当于在 `$NEMU_HOME` 目录下使用了命令 `make ISA=riscv32 run ARGS=... IMG=...`，这样就实现了将镜像加载到 nemu 上运行。

以下是我的自由报告内容，我在做实验的过程中详细记录了自己遇到的所有问题和查阅的所有资料。这些笔记按照章节排列，每章通常有两个部分：

- 思考题：包含笔者针对思考题查阅的资料和我尝试给出的答案。
- 补充：这里的内容是没有在讲义中提及的问题，以及讲义中建议自学的东西。

不停计算的机器

必答题

理解YEMU如何执行程序

> YEMU可以看成是一个简化版的NEMU, 它们的原理是相通的, 因此你需要理解YEMU是如何执行程序的. 具体地, 你需要
>
> - 画出在YEMU上执行的加法程序的状态机
> - 通过RTFSC理解YEMU如何执行一条指令
>
> 思考一下, 以上两者有什么联系?

我们定义一个状态为 (PC, R[0], R[1], M[7], halt)，则状态机为：

```
(0, x, x, 0, 0) -> (0, 33, x, 0, 0) -> (1, 33, x, 0, 0) -> (1, 33, 33, 0, 0) ->
(2, 33, 33, 0, 0) -> (2, 16, 33, 0, 0) -> (3, 16, 33, 0, 0) -> (3, 49, 33, 0,
0) -> (4, 49, 33, 0, 0) -> (4, 49, 33, 49, 0) -> (5, 49, 33, 49, 0) -> (5, 49,
33, 49, 1)
```

YEMU 执行指令的流程为：

- 取指：将 $M[pc]$ 处的内容拿出来
- 译码：根据指令的各个字段判断指令到底想让计算机做什么
- 执行：位于 switch 代码段中，YEMU 支持的指令非常简单，只有寄存器赋值，加法，load/store 四种。
- 更新 PC

两者的联系在于：程序的两个状态之间的变化和指令的功能是一一对应的。

RTFSC(2)

必答题

RTFSC 理解指令执行的过程

> 这一小节的细节非常多, 你可能需要多次阅读讲义和代码才能理解每一处细节. 根据往届学长学姐的反馈, 一种有效的理解方法是通过做笔记的方式来整理这些细节. 事实上, 配合GDB食用效果更佳.
>
> 为了避免你长时间对代码的理解没有任何进展, 我们就增加一道必答题吧:
>
> > 请整理一条指令在NEMU中的执行过程.

> 除了`nemu/src/device`和`nemu/src/isa/$ISA/system`之外, NEMU的其它代码你都已经有能力理解了. 因此不要觉得讲义中没有提到的文件就不需要看, 尝试尽可能地理解每一处细节吧! 在你遇到bug的时候, 这些细节就会成为帮助你调试的线索.

`fetch_decode_exec_updatepc()` 函数分成三个部分：`fetch_decode()` 负责取指译码，`s->EHelper(s)` 利用函数指针调用了译码后指令应该执行的函数。`cpu->pc=s->dnpc` 将 pc 指向下一条指令。

取指

`fetch_decode()` 调用了 `isa_fetch_decode()`。`isa_fetch_decode` 会调用 `instr_fetch()` 把指令从内存中取出来。`instr_fetch(*pc, len)` 的更深内容和物理内存读写有关，此处不深究。特别的一点是它取出指令后会根据 len 更新 pc 的值（指向下一条指令）。

译码

`isa_fetch_decode()` 将 `instr_fetch()` 的返回结果存储到 `s->instr.val` 中。`s` 是一个 `Decode` 结构体类型，其中的 `instr` 是 `ISADecodeInfo` 类型。

> 这里有一个有趣的代码细节是 nemu 为了抽象化不同 ISA 的差异，在顶层的代码中使用的是相同的类型名称，这些名称在底层才会分出区别，例如 `ISADecodeInfo` 类型的定义是
>
> ```
> typedef concat(__GUEST_ISA__, _ISADecodeInfo) ISADecodeInfo;
> ```
>
> `concat` 函数的功能是将两个字符串拼接起来，从而实现了根据选择的 ISA 的不同定义不同的 `ISADecodeInfo`，将 ISA 的差异对上层抽象了。

在本实验中 `instr` 自然是 `riscv32_ISADecodeInfo` 类型。该类型的定义可以在 `isa-def.h` 中查到，这是一个 union，给 `instr.val` 赋值之后，我们可以通过调用 union 中的各个变量来轻松获得指令的各个部分。

nemu 调用了 `table_main` 函数来进行指令类型的确认，所有的 table 函数可以被看做一个巨大的 switch-case。这里面有两种类型的宏：

```
def_INSTR_IDTAB("??????? ????? ????? ??? ????? xxxxx xx", I/U/S/J/B/R     ,
xxx);
def_INSTR_TAB("??????? ????? ????? ??? ????? xxxxx xx", xxx)
```

第一种和第二种的区别在于第一种还会按照传送的指令类型对指令的寄存器和立即数进行解析。这种宏会返回一个执行函数的编号。类型确认的过程无非是一些位运算，这里主要再看寄存器和立即数的解析：

为了进一步实现指令译码和操作数译码的解耦，nemu 还定义了译码操作数辅助函数，专门用于对立即数或寄存器进行译码。s 中有三个 Operand 类型的变量。Operand 类型的变量中有一个 union，分别是指向寄存器的指针，带符号立即数和无符号立即数，代表了寄存器类型/有符号立即数/无符号立即数。针对立即数的译码操作数辅助函数很简单，只要将值赋给 imm 即可。针对寄存器的则略微复杂一些：为了实现 riscv 中零号寄存器永远为0、不可写的特性，译码操作数辅助函数会对操作的读/写，和目标进行判断，以确保不会修改零号寄存器的内容 (is_write 参数只在寄存器中才会有作用）。

所有的译码辅助函数都是可以通过调用译码操作数辅助函数来高效完成。这部分和手册可以形成高度的对应。

执行

在 cpu-exec 中执行的代码很简单：就是调用 s 中存好的执行函数。形如 `exec_xxx` 的执行函数都是通过更加基本的 rtl 指令来实现功能。rtl 指令大多是调用了基于 c 语言的表达式运算来实现功能，少数的跳转指令稍微复杂一些。

更新 pc

将 `s->dnpc` 赋给 `s`，这里要注意 `dnpc` 和 `snpc` 的区别：`snpc` 仅仅指内存中该条指令的下一条指令（pc+4）位置，而 `dnpc` 保存了下一条真正要执行的指令的位置。在一些跳转指令中 `snpc` 和 `dnpc` 不一定相同。

思考题

立即数背后的故事

> 框架代码通过`instr_fetch()`函数进行取指, 别看这里就这么一行代码, 其实背后隐藏着针对字节序的慎重考虑. 大部分同学的主机都是x86小端机, 当你使用高级语言或者汇编语言写了一个32位常数`0x1234`的时候, 在生成的二进制代码中, 这个常数对应的字节序列如下(假设这个常数在内存中的起始地址是x):
>
> ```
> x   x+1  x+2  x+3+----+----+----+----+| 34 | 12 | 00 | 00 |+----+----+----
> +----+
> ```
>
> 而大多数PC机都是小端架构(我们相信没有同学会使用IBM大型机来做PA), 当NEMU运行的时候,
>
> ```
> imm = instr_fetch(pc, 4);
> ```
>
> 这行代码会将`34 12 00 00`这个字节序列原封不动地从内存读入`imm`变量中, 主机的CPU会按照小端方式来解释这一字节序列, 于是会得到`0x1234`, 符合我们的预期结果.
>
> Motorola 68k系列的处理器都是大端架构的. 现在问题来了, 考虑以下两种情况:
>
> - 假设我们需要将NEMU运行在Motorola 68k的机器上(把NEMU的源代码编译成Motorola 68k的机器码)
> - 假设我们需要把Motorola 68k作为一个新的ISA加入到NEMU中
>
> 在这两种情况下, 你需要注意些什么问题? 为什么会产生这些问题? 怎么解决它们?
>
> 事实上不仅仅是立即数的访问, 长度大于1字节的内存访问都需要考虑类似的问题. 我们在这里把问题统一抛出来, 以后就不再单独讨论了.

如果我们要将 NEMU 运行在 Motorola 68k 机器上，那么我们访问内存的指令就不能像现在这样将连续4个字节直接读入变量，因为 Motorola 68k 会按照大端方式来读取这4个字节。我们需要自己手写一个循环来按照小端方式处理数据。

如果我们要在 NEMU 中加入 Motorola 68k 的 ISA，那么相似地，我们不能将连续4个字节直接读入变量，而是要手写一个循环按照大端方式处理数据。

立即数背后的故事(2)

> mips32和riscv32的指令长度只有32位, 因此它们不能像x86那样, 把C代码中的32位常数直接编码到一条指令中. 思考一下, mips32和riscv32应该如何解决这个问题?

可以设置两种加载立即数的指令：一种向高位加载，一种向低位加载。riscv32 中的 lui 指令可以实现向高位加载立即数的功能。mips指令集由于没有接触过，暂时留坑。

为什么不需要 `rtl_muls_lo`

> 我们没有定义用于获取有符号数乘法结果低32位的RTL基本指令`rtl_muls_lo`, 你知道为什么吗?

`rtl_muls_lo` 所需实现的功能与 `rtl_mulu_lo` 完全相同，本着不要让多个函数做同一件事情的原则，我们不需要 `rtl_muls_lo`。但有符号和无符号在高位的表现是不同的，因此两种 hi 我们都需要。

RTL寄存器中值的生存期

> 在程序设计课上, 我们知道C语言中不同的变量有不同的生存期: 有的变量的值会一直持续到程序结束, 但有的变量却很快消亡. 在上述定义的RTL寄存器中, 其实也有不同的生存期. 尝试根据生存期给RTL寄存器分类.
>
> 尽管目前这个分类结果并没有什么用处, 但其实将来在PA5中设计RTL优化方案的时候, 生存期的性质会给我们提供很大的优化机会.

留坑，二周目再做。

为什么执行了未实现指令会出现上述报错信息

> RTFSC, 理解执行未实现指令的时候, NEMU具体会怎么做.

在执行了未实现的指令时，译码的过程中无法匹配到任何一种 pattern，最终会返回一个 `EXEC_ID_inv` 宏，即不合法指令对应的执行函数的编号。这里的宏利用元编程和 enum 的方法写出了一种鲁棒性很强的代码，即使不断添加新指令也可以使得 `EXEC_ID_inv` 代表的编号是正确的。

拿到 `EXEC_ID_inv` 后，执行函数赋为 `g[idx]`。在执行 `g[idx]` 时，执行的是 `exec_inv`，`exec_inv()` 函数调用了 `rtl_hostcall` 函数，并传入了 `HOSTCALL_INV` 参数（与正常退出的 `HOSTCALL_EXIT` 区分）。`rtl_hostcall` 函数接收到 `HOSTCALL_INV` 参数时，便会打印上述的报错信息。

程序，运行时环境与AM

思考题

这又能怎么样呢

> 思考一下, 这样的抽象还会带来哪些好处呢? 你很快就会体会到这些好处了.

这样的抽象有利于代码的移植：你甚至可以把别人的上层代码拿来直接用，即使你们使用不同的语言做了 nemu，只要 API 接口统一，上层的操作系统和用户程序就可以调用 API 直接运行。

[二周目]为什么要有AM？

> 操作系统也有自己的运行时环境. AM和操作系统提供的运行时环境有什么不同呢? 为什么会有这些不同?

留坑。笔者暂时认为 AM 中的运行时环境更多是对硬件功能的直接抽象：比如可以访问外设，可以读写内存等。而操作系统中的运行时环境抽象层次更高，比如系统调用等等，是对 AM 运行时环境的进一步封装。之所以不同是因为他们所处的抽象层不同，服务的对象也不同。操作系统向应用程序暴露 API，自然不需要让应用程序知道硬件细节。AM 的运行时环境可以让操作系统更好地利用已有的硬件功能，可以做到架构和OS的解耦。

mips32的分支延迟槽

> 为了提升处理器的性能, mips使用了一种叫分支延迟槽的技术. 采用这种技术之后, 程序的执行顺序会发生一些改变: 我们把紧跟在跳转指令(包括有条件和无条件)之后的静态指令称为延迟槽, 那么程序在执行完跳转指令后, 会先执行延迟槽中的指令, 再执行位于跳转目标的指令. 例如
>
> ```
> 100: beq 200101: add102: xor...200: sub201: j   102202: slt
> ```
>
> 若`beq`指令的执行结果为跳转, 则相应的动态指令流为`100 -> 101 -> 200`; 若`beq`指令的执行结果为不跳转, 则相应的动态指令流为`100 -> 101 -> 102`; 而对于`j`指令, 相应的动态指令流为`201 -> 202 -> 102`.
>
> 你一定会对这种反直觉的技术如何提升处理器性能而感到疑惑. 不过这需要你先了解一些微结构的知识, 例如处理器流水线, 但这已经超出了ICS的课程范围了, 所以我们也不详细解释了, 感兴趣的话可以STFW.
>
> 但我们可以知道, 延迟槽技术需要软硬件协同才能正确工作: mips手册中描述了这一约定, 处理器设计者按照这一约定设计处理器, 而编译器开发者则会让编译器负责在延迟槽中放置一条有意义的指令, 使得无论是否跳转, 按照这一约定的执行顺序都能得到正确的执行结果.
>
> 如果你是编译器开发者, 你将会如何寻找合适的指令放到延迟槽中呢?

流水线CPU中如果分支预测失败，常见的做法是对各个流水段进行冲刷，然后将应该跳转到的PC送入流水线继续执行。flush 是一个代价很大的操作，会使流水线浪费很多个时钟周期，因此我们有两个方向可以努力：一个是尽可能减少 flush 的次数，也就是提高分支预测的成功率（动态分支预测 etc.），另一个是使流水线中执行的指令即使在分支预测错误的时候也可以保留，这就是mips中的延迟槽。

至于分支延迟槽中应该放什么指令，最简单的自然是插入nop指令。这样正确性无疑可以得到保证，但性能不高：不论分支预测是否成功，执行这条nop指令都相当于没做事情，可以理解为流水线被阻塞了。

以下是笔者的一些猜想：如果跳转和不跳转的接下来的几条指令中有完全相同的指令，自然可以将这条指令放到延迟槽中。这样不论走了哪个分支这条指令的执行都是有用的。当然，能否调整指令的顺序取决于指令之间的依赖关系。不过完全相同的指令很难找，我们可以利用寄存器的重命名技术，现将某条指令的结果放到一个备用的内存空间中，如果将来走这条分支，就将对应的寄存器名映射到这个地址上。

指令名对照

> AT&T格式反汇编结果中的少量指令, 与手册中列出的指令名称不符, 如x86的`cltd`, mips32和riscv32则有不少伪指令(pseudo instruction). 除了STFW之外, 你有办法在手册中找到对应的指令吗? 如果有的话, 为什么这个办法是有效的呢?

笔者使用的一个方法是：直接将这条伪指令对应的机器代码二进制串放进手册里搜索。因为伪指令实际上也是通过普通指令完成功能的，所以可以搜索到对应的普通指令，从而确定伪指令的行为。

stdarg是如何实现的?

> `stdarg.h`中包含一些获取函数调用参数的宏, 它们可以看做是调用约定中关于参数传递方式的抽象. 不同ISA的ABI规范会定义不同的函数参数传递方式, 如果让你来实现这些宏, 你会如何实现?

笔者并没有去看 `stdarg.h` 的源码，但笔者用过宏 `__VA_ARGS__`，认为它背后的实现方式应该是一个链表，链表中的每个元素包含了参数的名称和参数指向的地址。

补充

关于 riscv 的 jalr 指令

笔者在实现 jalr 指令的时候犯了一些错误，这个错误直到 PA3 才暴露出来，在此特地记录。jalr 指令的行为在手册中的定义如下：

```
t=pc+4; pc=(x[rs1]+sext(offset))&~1; x[rd]=t
```

笔者起初认为这样的写法非常麻烦：为什么不把 pc+4 的值直接赋给 x[rd]，而要搞出这么一个中间变量 t 呢？事实上这是因为指令中 rs1 和 rd 可能是一样的，如果在第一条指令就直接修改了 x[rd]，就可能也修改了 x[rs1]，从而第二条指令的结果就不对了。因此中间变量 t 的设置绝非是画蛇添足。

ISA 的手册经过了很多人的打磨，是非常简练的。每一条看似无用的语句一定都是充分考虑到了一些特殊情况无法删除才呈现在手册中的。因此最简单的保证正确的实现方法就是：**手册说什么你就写什么**。

基础设施(2)

思考题

消失的符号

> 我们在`am-kernels/tests/cpu-tests/tests/add.c`中定义了宏`NR_DATA`, 同时也在`add()`函数中定义了局部变量`c`和形参`a`, `b`, 但你会发现在符号表中找不到和它们对应的表项, 为什么会这样? 思考一下, 什么才算是一个符号(symbol)?

宏只是做了简单的字符串替换，在预处理阶段就会处理掉，显然不会进入符号表。局部变量和形式参数都是在栈上定义的，所以不在符号表中。只有全局变量、函数、静态全局变量、只读字符串等才会出现在符号表中。

寻找"Hello World!"

> 在Linux下编写一个Hello World程序, 编译后通过上述方法找到ELF文件的字符串表, 你发现"Hello World!"字符串在字符串表中的什么位置? 为什么会这样?

笔者先用命令 `hd a.out` 查看二进制文件并找到了 "Hello, World\n" 字符串在文件中的偏移量，然后用 `readelf -a a.out` 命令查看了所有节，发现该字符串在只读数据节中。之所以在只读代码节中是因为这个字符串是理论上不应该被修改的。

不匹配的函数调用和返回

> 如果你仔细观察上文`recursion`的示例输出, 你会发现一些有趣的现象. 具体地, 注释(1)处的`ret`的函数是和对应的`call`匹配的, 也就是说, `call`调用了`f2`, 而与之对应的`ret`也是从`f2`返回; 但注释(2)所指示的一组`call`和`ret`的情况却有所不同, `call`调用了`f1`, 但却从`f0`返回; 注释(3)所指示的一组`call`和`ret`也出现了类似的现象, `call`调用了`f1`, 但却从`f3`返回.
>
> 尝试结合反汇编结果, 分析为什么会出现这一现象.

这是一个非常有意思的问题，我们以注释(2)处的代码为例分析这一情况：

注释(2)处显示，函数 `f2` 中调用了 `f1`，但没有从 `f1` 返回而是从 `f0` 返回的。使用 nemu 追踪 CPU 是如何从 `f1` 悄悄跳到 `f0` 的，发现了 `f1` 结尾处的如下汇编代码：

```
80000090:   00158593            addi    a1,a1,180000094:    fff50513
addi    a0,a0,-180000098:   00078067            jr  a58000009c: 00100513
     li  a0,1800000a0:   00008067            ret
```

`f1` 的 C 语言原文如下：

```
int f1(int n, int l) {  if (l > lvl) lvl = l;  rec ++;  return n <= 0 ? 1 :
func[0](n - 1, l + 1);};
```

正是位于地址 0x80000098 处的指令 `jr a5` 从 `f1` 跳转到了 `f0`，也就是说其实我们的 ftrace 追踪漏了一次调用。这一跳转指令之所以会在追踪中被遗漏是因为它不符合标准的跳转指令的形式：标准的跳转指令一般都是 `jal` 或 `jalr` 指令，且存储 snpc 的寄存器按照 calling convention 应为 `ra`，以方便 return 回来以后从跳转指令的下一条开始继续执行。这里使用 `jr a5` 直接跳转，会导致从 `f0` 返回后其实没有回到 `f1`。

这里之所以出现这样的情况是因为编译器对**尾调用**进行了优化。因为 `f1` 调用 `f0` 在 `f1` 的函数末尾，所以执行完 `f0` 返回之后 `f1` 会立即拿着 `f0` 的返回值再返回给 `f2`，所以不妨让 `f0` 直接返回 `f2`。这个优化在尾递归中会格外有用，不仅省去了一些指令，还可以重复利用栈帧，使得层数很深的尾递归也不会爆栈。

冗余的符号表

> 在Linux下编写一个Hello World程序, 然后使用`strip`命令丢弃可执行文件中的符号表:
>
> ```
> gcc -o hello hello.cstrip -s hello
> ```
>
> 用`readelf`查看hello的信息, 你会发现符号表被丢弃了, 此时的hello程序能成功运行吗?
>
> 目标文件中也有符号表, 我们同样可以丢弃它:
>
> ```
> gcc -c hello.cstrip -s hello.o
> ```
>
> 用`readelf`查看hello.o的信息, 你会发现符号表被丢弃了. 尝试对hello.o进行链接:
>
> ```
> gcc -o hello hello.o
> ```
>
> 你发现了什么问题? 尝试对比上述两种情况, 并分析其中的原因.

第一种情况丢弃了符号表之后仍然可以正常运行，第二种情况丢弃了符号表后会报错：

```
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/10/../../../x86_64-linux-
gnu/Scrt1.o: in function `_start':(.text+0x24): undefined reference to `main'
```

因为符号表被删除了，无法找到 main 函数的位置，所以链接失败。符号表的作用就是在链接的时候告诉外部模块本模块的各个函数、变量的地址，在已经生成好的可执行文件中，所有重定位已经完成，自然不再需要符号表；但如果在可重定位文件中删去了符号表，链接则会出问题。

如何生成 native 的可执行文件

> 阅读相关Makefile, 尝试理解`abstract-machine`是如何生成`native`的可执行文件的.

Makefile 中有一个变量 `CROSS_COMPILE`，在 native 下运行的时候这个变量没有赋值，所以 Makefile 不会使用交叉编译工具编译代码，而是用 native 的 gcc 和 binutils。

奇怪的错误码

> 为什么错误码是`1`呢? 你知道`make`程序是如何得到这个错误码的吗?

如果 main 函数正常退出，返回值会是0，否则返回值是1。`make` 程序捕捉 main 函数的返回值，并据此判断是否发生了错误，并将错误码呈现出来。

这是如何实现的？

> 为什么定义宏`__NATIVE_USE_KLIB__`之后就可以把`native`上的这些库函数链接到klib? 这具体是如何发生的? 尝试根据你在课堂上学习的链接相关的知识解释这一现象.

klib 中编写的所有库函数都被条件编译 ifdef 框了起来，参与编译的条件为 `!defined(__ISA_NATIVE__) || defined(__NATIVE_USE_KLIB))`。如果 `NATIVE_USE_KLIB` 被注释了的话，在 native 下运行就不会有手写的库函数，native 会使用 glibc 的库函数。如果 `NATIVE_USE_KLIB` 没有被注释，在 native 下运行有手写的库函数，当手写函数与 glibc 库函数名称冲突时，优先链接用户手写的函数，这样就达到了使用 klib 的效果。

匪夷所思的QEMU行为(有点难度)

> 在一些旧版的mips32-QEMU中, 仅在上述指令的PC值后12位为`0xffc`时, 才会进行指令打包. 这个打包条件看上去非常奇怪, 你知道可能的原因是什么吗?

笔者没有使用 QEMU 进行 difftest，也不了解 mips 指令集，故留坑。

捕捉死循环(有点难度)

> NEMU除了作为模拟器之外, 还具有简单的调试功能, 可以设置断点, 查看程序状态. 如果让你为NEMU添加如下功能
>
> > 当用户程序陷入死循环时, 让用户程序暂停下来, 并输出相应的提示信息
>
> 你觉得应该如何实现? 如果你感到疑惑, 在互联网上搜索相关信息.

从理论上证明某个程序会死循环是不可能的，因为这涉及到图灵停机问题：比如我写一个 check 哥德巴赫猜想的程序，如果我能判断它是否会死循环，我就证明了哥德巴赫猜想。

笔者结合了网上的资料并自己思考了之后认为可以有如下的一些捕捉方法：

- 在一个进程运行的同时用另一个进程来监视它的行为，如果该进程耗时过长就将其 kill。
- 死循环主要来自 do-while 和 for 循环。虽然无限递归从理论上也会死循环，但硬件的限制会使其通过爆栈的方式停下来。因此可以在编译是找到所有的 do-while 和 for 循环，向其中加入计数器，在循环执行次数超过了一个阈值后强行退出。

补充

关于 Difftest

笔者刚开始无法通过 spike 的 difftest，查看寄存器状态发现 `t0` 寄存器的值不正确。产生这个问题的原因是笔者在实现 jal/jalr 指令时使用了 `t0` 寄存器保存中间结果——这一行为虽然符合讲义中的调用约定，但由于 spike 没有相同的行为，所以报错了。事实上我们写的 nemu 并不是真正的“硬件”：我们根本不需要真的用寄存器来保存中间结果——我们有 C 语言，定义一个变量保存就可以了！框架代码中给的 `rtl_j` 函数似乎也使用了类似的手法（笔者没有调用框架代码给的 `rtl_j` 来实现跳转指令）

输入输出

必答题

游戏是如何运行的

> 请你以打字小游戏为例, 结合"程序在计算机上运行"的两个视角, 来剖析打字小游戏究竟是如何在计算机上运行的. 具体地, 当你按下一个字母并命中的时候, 整个计算机系统(NEMU, ISA, AM, 运行时环境, 程序) 是如何协同工作, 从而让打字小游戏实现出"命中"的游戏效果?

在主循环之间，打字小游戏做了一些绘制屏幕方面的准备工作。这些工作在 `vedio_init` 中，具体为

- 读取窗口的宽和高。程序读取窗口大小通过读取 AM 提供的 `AM_GPU_CONFIG` 抽象寄存器实现。`AM_GPU_CONFIG` 抽象寄存器则通过访问 nemu 提供的 I/O 端口来获得这些参数。
- 将整个画布涂成紫色的。程序向画布输出颜色通过向 AM 提供的 `AM_GPU_FBDRAW` 抽象寄存器写入内容来实现。抽象寄存器会接收到开始绘制的位置的坐标，绘制内容数量，绘制的内容的 buffer，和一个是否立即同步的布尔变量。抽象寄存器的行为很简单，就是将对应的数据写到 nemu 的 I/O 端口中。nemu 的硬件会每隔一段时间检查是否有同步信号，如果有就将缓冲区的内容输出到屏幕上。
- 对各种字母的颜色做好设定。

打字小游戏的主循环是一个 `while(1) {}` ，在循环中会做这样一些事情：

- `game_logic_update`：每次循环中都会根据和上次更新的时间差进行若干次游戏逻辑更新。这里程序调用时间函数会使用 AM 提供的抽象寄存器 `AM_TIMER_UPTIME` 来读取游戏已经开始的时间。AM中读取这个抽象寄存器的逻辑是访问 nemu 的时钟 I/O 端口。

  游戏逻辑更新中会定期生成一个新的字母，并且将每个字母根据其信息更新位置。关于字母，每个字母有如下一些参数：

  - `ch`：表示这个字母是啥，可以取 `A-Z`。
  - `(x,y)`：表示字母当前的位置。`x` 值是生成字母的时候随机的，`y` 值则会根据当前的时间不断更新。
  - `v`：表示字母单位时间移动的速度。正常情况下速度是正数，字母会往下落。当字母被击中的时候速度会变成负数，从而实现字母向上升的效果。如果速度为0，表示当前字母 miss 了。
  - `t`：这是一个计时器，用于当一个字母 miss 的时候，延迟 FPS 的时间后才会消失。
- 键盘读取：除非读取到 `AM_KEY_NONE` 表示当前没有按键，否则程序会一直在键盘读取的 while 循环中。这里程序收集键盘按键会使用 AM 提供的抽象寄存器 `AM_INPUT_KEYBRD`。`AM_INPUT_KEYBRD` 会访问 nemu 的键盘 I/O 端口，并根据得到的数据生成 keydown 和 keycode 两个参数。

  键盘读取分为以下两种情况：

  - 按下了 escape 键：调用 `halt(0)`，退出。
  - 按下了是字母的键：调用 `check_hit` 函数检查是否确实击中了字母。`check_hit` 的行为比较简单：如果没有匹配到任何一个字母则更新 wrong，否则会将击中的字母的速度改成一个负值，从而一面实现字母被击中的标记，一面实现字母向上升的效果。
- `render` ：这个函数负责更新屏幕。该函数会将字母原本所处位置的屏幕内容抹去（变成紫色），然后根据字母新的位置以及字母当前的状态（下落白色，miss红色，击中绿色）选择相应的 texture 作为 buffer 输出到对应位置。

思考题

理解 volatile 关键字

> 也许你从来都没听说过C语言中有`volatile`这个关键字, 但它从C语言诞生开始就一直存在. `volatile`关键字的作用十分特别, 它的作用是避免编译器对相应代码进行优化. 你应该动手体会一下`volatile`的作用, 在GNU/Linux下编写以下代码:
>
> ```
> void fun() {extern unsigned char _end;  // _end是什么?volatile unsigned char *p = &_end;*p = 0;while(*p != 0xff);*p = 0x33;*p = 0x34;*p = 0x86;}
> ```
>
> 然后使用`-O2`编译代码. 尝试去掉代码中的`volatile`关键字, 重新使用`-O2`编译, 并对比去掉`volatile`前后反汇编结果的不同.
>
> 你或许会感到疑惑, 代码优化不是一件好事情吗? 为什么会有`volatile`这种奇葩的存在? 思考一下, 如果代码中`p`指向的地址最终被映射到一个设备寄存器, 去掉`volatile`可能会带来什么问题?

`_end` 是一个变量，它的地址指向了数据段的第一个未使用的位置。

在有 `volatile` 关键字的情况下，反汇编结果为：

```
0000000000000000 <fun>:   0:    f3 0f 1e fa             endbr64    4:   c6 05
00 00 00 00 00    movb   $0x0,0x0(%rip)        # b <fun+0xb>   b: 48 8d 15 00
00 00 00    lea    0x0(%rip),%rdx        # 12 <fun+0x12>  12:   66 0f 1f 44 00
00       nopw   0x0(%rax,%rax,1)  18:    0f b6 02                movzbl
(%rdx),%eax  1b: 3c ff                   cmp    $0xff,%al  1d:   75 f9
          jne    18 <fun+0x18>  1f:   c6 05 00 00 00 00 33    movb
$0x33,0x0(%rip)        # 26 <fun+0x26>  26:  c6 05 00 00 00 00 34    movb
$0x34,0x0(%rip)        # 2d <fun+0x2d>  2d:  c6 05 00 00 00 00 86    movb
$0x86,0x0(%rip)        # 34 <fun+0x34>  34:  c3                      ret
```

可以看到这段指令和 C 语言程序的功能基本是一一对应的。

在没有 `volatile` 关键字的情况下，反汇编结果为：

```
0000000000000000 <fun>:   0:    f3 0f 1e fa             endbr64    4:   c6 05
00 00 00 00 00    movb   $0x0,0x0(%rip)        # b <fun+0xb>   b: eb fe
          jmp    b <fun+0xb>
```

指令发生了很大的变化。编译器发现 `*p=0` 之后进入了一个只有 `*p=0xff` 才会退出的循环，这期间没有任何对 p 的更改，所以认为这是一个死循环，直接将其缩减为了 `jmp b` ，并且砍掉了后面的所有内容。

如果 p 指向的地址被映射到了一个设备寄存器，那么原本这个程序的功能可能是正确的：当检测到设备寄存器的值变为 0xff 后，依次向设备寄存器写入 0x33, 0x34 和 0x86。但编译优化之后，这个程序就完全变成了一个死循环。因此我们并不是什么时候都希望激进的编译优化：编译器有可能考虑不到外设的参与等情况。

如何检测多个键同时被按下?

> 在游戏中, 很多时候需要判断玩家是否同时按下了多个键, 例如RPG游戏中的八方向行走, 格斗游戏中的组合招式等等. 根据键盘码的特性, 你知道这些功能是如何实现的吗?

键盘码是一个8位二进制数，如果我们用32位整数来传递键盘码，就可以同时传递4个，可以用于检测多个键同时按下。

神奇的调色板

> 现代的显示器一般都支持24位的颜色(R, G, B各占8个bit, 共有`2^8*2^8*2^8`约1600万种颜色), 为了让屏幕显示不同的颜色成为可能, 在8位颜色深度时会使用调色板的概念. 调色板是一个颜色信息的数组, 每一个元素占4个字节, 分别代表R(red), G(green), B(blue), A(alpha)的值. 引入了调色板的概念之后, 一个像素存储的就不再是颜色的信息, 而是一个调色板的索引: 具体来说, 要得到一个像素的颜色信息, 就要把它的值当作下标, 在调色板这个数组中做下标运算, 取出相应的颜色信息. 因此, 只要使用不同的调色板, 就可以在不同的时刻使用不同的256种颜色了.
>
> 在一些90年代的游戏中(比如仙剑奇侠传), 很多渐出渐入效果都是通过调色板实现的, 聪明的你知道其中的玄机吗?

在保持颜色索引不变的情况下，只要平滑地切换调色板就可以实现渐变。如果准备多套调色板，事实上我们可以使用远不止256种颜色。

LiteNES 如何工作？

> 另一个值得RTFSC的项目是LiteNES, 除了内置rom之外, 代码总数大约1500行. 关键是这个小巧玲珑的项目里面已经包含了一个完整的计算机系统: CPU, 内存, MMIO, 以及手柄(psg), 卡带(mmc)和图形处理器(ppu)这三个外设. 除了ppu的内部实现细节之外, 其余的部分你都已经有能力去理解了.
>
> 有趣的是, LiteNES可以看成是NEMU和AM程序的融合. 尝试阅读LiteNES的代码, 理解LiteNES作为一个完整的计算机系统, 上述部件如何交互, 以及LiteNES作为一个AM程序, 如何通过AM提供的API来实现游戏效果. 我们提供6502处理器(NES的CPU)以及NES PPU(图形处理器)的资料供大家参考.

留坑。

补充

关于串口背后的原理

> `putch` 仅仅执行了 `outb(SERIAL_PORT,ch)`，而 `outb(SERIAL_PORT,ch)` 仅仅将 *SERIAL_PORT 的值赋成了 ch，为什么就能同时通过标准错误流把信息输出在屏幕上呢？

我们需要深入到汇编层面理解这段代码，`putch` 编译成汇编代码后仅有三句话：

```
lui a5, 0xa0000sb a0, 1016(a5)ret
```

第一句话装载立即数，结合第二句的 1016(a5) 定位到 SERIAL_PORT 指向的地址，第二句话就是真正的写入函数。虽然看上去仅仅是 store，但向 I/O 设备的 store 行为是更丰富的，在 nemu 中它会走过这样的流程：

- 取指译码之后获取所有的信息，并分配好一个存储相关的执行函数。
- 运行执行函数，store 的执行函数会调用 `paddr_write`。
- `paddr_write` 分析了一下要写入的地址，发现并不在内存中，而是在 I/O 区域，于是调用了 `mmio_write`。
- `mmio_write` 分析了一下要写入的地址，发现是串口的地址，于是调用了 `map_write`，并把串口的 IOMap 传了过去。
- `map_write` 做了一些正确性检查之后，先调用了 `host_write`，将 ch 这个数据写到了 SERIAL_PORT 地址，然后调用了串口的回调函数。
- 串口的回调函数是 `serial_io_handler`，串口的回调函数默认是输出，否则会报错，在 `is_write==true` 的情况下回调函数会调用 `serial_putc`。
- `serial_putc` 使用了 glibc 中的 `putc(ch,stderr)` 将字符输出到了标准错误流。

因此事实上虽然从高级语言层面 `putch` 中只有一个存储指令，但在汇编层面它既完成了存储又完成了输出。符合串口的 feature。

疑问

如果 nemu 所有向屏幕输出的函数最终都是通过 putch 来实现，那么按理来说只需要申请一个字节大小的串口空间就可以了，为什么要 8 个字节呢？

关于时钟硬件接口

框架代码中有一个小细节：只有在读取时钟的高32位的时候才会触发 nemu 重新读取当前时间。这意味着如果需要读取时间，一定要先读取高32位再读取低32位，顺序反过来就会出错。这样有一个好处：因为两次读取总是有时间差的，所以这样可以保证高32位和低32位来自同一个时刻。

声卡

笔者完成了声卡的选做题。声卡的 I/O 相较于其他的会复杂一些，因为 nemu 的硬件部分的代码也需要自己实现。nemu 不是真正的“硬件”，要实现音频的输出还是得靠库函数。这里笔者根据讲义使用了 SDL 库。

我们需要在 AM 中实现以下函数：

- `__am_audio_config`：读取声卡的状态，包括是否存在声卡 (cfg->present)。
- `__am_audio_ctrl`：告诉硬件一些基本的信息，如频率，频道，采样率等等。
- `__am_audio_status`：读取缓冲区中还有的数据字节数。
- `__am_audio_play`：向声卡的缓冲区写入数据。如果声卡的缓冲区不够，则应该轮询，直到缓冲区大小足够位置。

我们需要在 nemu 中做这样一些事情：

- 在 audio 的回调函数 `audio_io_handler` 中完成初始化，包括向 `SDL_AudioSpec` 类型的变量中填写频率、频道、采样率，注册一个回调函数 `audio_callback()`，以及调用 `SDL_InitSubSystem` 和 `SDL_OpenAudio` 函数等。
- 完成 audio-sbuf 的回调函数 `audio_sbuf_handler`，主要工作就是把 AM 送过来的数据放入缓冲队列。
- 完成 `audio_callback()`，SDL 会不定时索要一定量的数据，从当前缓冲区的队列头部取数据给 SDL。如果当前所剩数据量不够，应该再补充一些 0，以防出现噪音。

PA3 - 穿越时空的旅程: 批处理系统

实验进度

我完成了所有的必答题，并针对选做题和思考题给出了自己的想法。

必答题

理解上下文结构体的前世今生

点击这里跳转到解答。

理解穿越时空的旅程

点击这里跳转到解答。

hello程序是什么, 它从何而来, 要到哪里去

点击这里跳转到解答

仙剑奇侠传究竟如何运行

> 运行仙剑奇侠传时会播放启动动画, 动画里仙鹤在群山中飞过. 这一动画是通过`navy-apps/apps/pal/repo/src/main.c`中的`PAL_SplashScreen()`函数播放的. 阅读这一函数, 可以得知仙鹤的像素信息存放在数据文件`mgo.mkf`中. 请回答以下问题: 库函数, libos, Nanos-lite, AM, NEMU是如何相互协助, 来帮助仙剑奇侠传的代码从`mgo.mkf`文件中读出仙鹤的像素信息, 并且更新到屏幕上? 换一种PA的经典问法: 这个过程究竟经历了些什么? (Hint: 合理使用各种trace工具, 可以帮助你更容易地理解仙剑奇侠传的行为)

程序的行为总体上是读出仙鹤的像素信息，然后不断更新到屏幕上并修改仙鹤的位置。在软件层面，程序会使用fseek 和 fread 库函数来定位和读取像素信息，使用SDL库函数来输出。fseek 和 fread 函数最终会使用`lseek` 和 `read` 系统调用，SDL库函数基于我们对系统调用进一步封装的NDL库实现，NDL库会使用系统调用 `lseek` 和 `write` 来实现像素的定位和写入（注意VGA被抽象成了文件，读写像素不需要专门的 I/O 相关函数。）`lseek` `read` 和 `write` 最终会调用 libos 中的 _read, _write 和 _lseek，传递对应的系统调用号和相关参数，使用 `ecall` 指令触发中断。

Nanos-lite 中会识别系统调用的类型，并调用 `fs_read` `fs_write` 和 `fs_lseek` 。`fs_lseek` 比较简单，只是修改文件当前的 offset，`fs_read` 会使用 `/bin/pal` 对应的读函数 `ramdisk_read` 来从“硬盘”中读取信息（事实上已经被加载到内存中）`ramdisk_read` 使用的是 AM 的函数 memcpy；`fs_write` 会使用 `/dev/fb` 文件对应的写函数 `fb_write` 。`fb_write` 会使用 AM 提供的 VGA 相关的抽象寄存器和一段像素空间来写入信息。

AM 和 nemu 之间有一套内存 I/O 的映射规定，nemu 会根据这些 I/O 空间的信息进行硬件层面的修改。memcpy 是 klib 中的函数，可以直接访问 nemu 的大数组（内存），实现信息的读取。

以下是我的自由报告内容，我在做实验的过程中详细记录了自己遇到的所有问题和查阅的所有资料。这些笔记按照章节排列，每章通常有两个部分：

- 思考题：包含笔者针对思考题查阅的资料和我尝试给出的答案。
- 补充：这里的内容是没有在讲义中提及的问题，以及讲义中建议自学的东西。

最简单的操作系统

思考题

[二周目] 什么是操作系统？

> 这可是个大问题, 我们也鼓励你学习完操作系统课程之后再回来重新审视它.

留坑，二周目再填。目前笔者认为操作系统是直接与硬件接触的一个程序，它将硬件支持的功能封装成系统调用作为服务提供给上层应用程序，并负责上层应用程序之间的运行调度。

补充

初次运行 nanos-lite 报错

初次运行 nanos-lite 显示程序访问了非法的地址，经过定位后发现 AM 中的 printf 没有实现 `%p` 打印指针的功能，触发了 UB。这为我们敲响了编写好的代码的警钟：如果函数的参数输入出现了“未定义“的行为，函数应该输出一些提示信息，从而快速定位问题。（例如此处就应该让 printf 报错，而不是通过非法的地址访问去寻找 fault）。

穿越时空的旅程

必答题

理解上下文结构体的前世今生

> 你会在`__am_irq_handle()`中看到有一个上下文结构指针`c`, `c`指向的上下文结构究竟在哪里? 这个上下文结构又是怎么来的? 具体地, 这个上下文结构有很多成员, 每一个成员究竟在哪里赋值的? `$ISA-nemu.h`, `trap.S`, 上述讲义文字, 以及你刚刚在NEMU中实现的新指令, 这四部分内容又有什么联系?
>
> 如果你不是脑袋足够灵光, 还是不要眼睁睁地盯着代码看了, 理解程序的细节行为还是要从状态机视角入手.

`trap.S` 文件的前半部分负责组织上下文结构体。这个上下文结构体作为函数的参数，保存在栈上。`trap.S` 的行为和正常的 C 程序调用函数前准备参数的过程是一样的。第一句

```
addi sp, sp, -CONTEXT_SIZE
```

为 Context 结构体开辟了空间。接下来的

```
MAP(REGS, PUSH)
```

利用了一点元编程技巧，其含义是将通用寄存器的值按顺序上栈。

接下来还要将特权寄存器的值上栈。特权寄存器不能用 store 指令直接上栈，因此要先将其值保存到通用寄存器中再上栈。代码是如下6行：

```
csrr t0, mcause
csrr t1, mstatus
csrr t2, mepc

sd t0, OFFSET_CAUSE(sp)
sd t1, OFFSET_STATUS(sp)
sd t2, OFFSET_EPC(sp)
```

在进入处理函数之前，最后一句话是

```
mv a0, sp
```

根据 RISCV 的 calling convention，a0 寄存器保存了第一个参数的值。我们后面要调用的函数 `__am_irq_handle` 的参数是 `Context *c`，因此将当前的 sp 值传进 a0，进入函数后指针 c 就指向了在栈上保存的上下文结构体的首地址。

这整个过程充分利用了 C 和汇编语言的紧密联系。

理解穿越时空的旅程

> 从Nanos-lite调用`yield()`开始, 到从`yield()`返回的期间, 这一趟旅程具体经历了什么? 软(AM, Nanos-lite)硬(NEMU)件是如何相互协助来完成这趟旅程的? 你需要解释这一过程中的每一处细节, 包括涉及的每一行汇编代码/C代码的行为, 尤其是一些比较关键的指令/变量. 事实上, 上文的必答题"理解上下文结构体的前世今生"已经涵盖了这趟旅程中的一部分, 你可以把它的回答包含进来.

> 别被"每一行代码"吓到了, 这个过程也就大约50行代码, 要完全理解透彻并不是不可能的. 我们之所以设置这道必答题, 是为了强迫你理解清楚这个过程中的每一处细节. 这一理解是如此重要, 以至于如果你缺少它, 接下来你面对bug几乎是束手无策.

Nanos-lite 调用 `yield()` 之后，执行了两条汇编指令（这两条汇编指令是用内联汇编的方式直接嵌入的）：

```
asm volatile("li a7, -1; ecall");
```

其中第一条指令向 `a7` 寄存器写入 -1，`a7` 寄存器是约定中传递中断类型的寄存器。第二条指令 `ecall` 则是“中断”指令。

在硬件层面，nemu 对 `ecall` 指令译码后，执行时会调用 `isa_raise_intr` 函数，该函数将 `a7` 寄存器传递的事件类型存放到 mcause 寄存器中，向 mepc 寄存器写入当前的 pc，然后将 pc 跳转到 mtvec 寄存器预先存好的一个地址开始执行。mtvec 中的地址是函数 `__am_asm_trap` 的地址，在 `cte_init` 中的

```
asm volatile("csrw mtvec, %0" : : "r"(__am_asm_trap));
```

完成了 mtvec 内容的初始化。

`__am_asm_trap` 函数在 `trap.S` 中定义，是用内联汇编写的。该函数首先组织上下文结构体，组织方式见上一道必答题。然后跳转进入处理函数 `__am_irq_handle`。函数 `__am_irq_handle` 会根据上下文结构体的内容打包出一个事件结构体 `ev`。根据上下文结构体中的 mcause 寄存器的值，可以识别事件的类型，如自陷事件的事件号为 -1。打包完事件结构体后，`__am_irq_handle` 会将上下文结构体和事件结构体一起传给处理函数。这个处理函数是在 `cte_init` 中传进来的 `do_event`。

`do_event` 函数检查事件结构体中的事件类型，当看到是 EVENT_YIELD 时，它判定这个中断没有必要重复，因此给 mepc 的值 +4，这个 +4 操作直接修改了上下文结构体中的 mepc 的保存值。

这时进入 `trap.S` 的后半部分，后半部分将栈上保存的寄存器内容恢复，然后调用 `mret` 指令。在硬件层面，nemu 识别出 `mret` 指令后，直接将 pc 恢复为 mepc 的值。至此，时空穿越的旅程结束。

思考题

[二周目] 特殊的原因？

> 这些程序状态(x86的eflags, cs, eip; mips32的epc, status, cause; riscv32的mepc, mstatus, mcause)必须由硬件来保存吗? 能否通过软件来保存? 为什么?

留坑，二周目再做。

异常号的保存

> x86通过软件来保存异常号, 没有类似cause的寄存器. mips32和riscv32也可以这样吗? 为什么?

x86的寄存器比较复杂，有类似于EFLAGS等大量的功能专一的寄存器，这些寄存器的值也是可以上栈的。而 riscv32 中只有通用寄存器的值可以上栈，如果一定要用软件来保存异常号的话，则必须硬件和软件约定好使用一个通用寄存器来专门保存异常号，这在本质上已经和cause寄存器没有区别了。

对比异常处理与函数调用

> 我们知道进行函数调用的时候也需要保存调用者的状态: 返回地址, 以及calling convention中需要调用者保存的寄存器. 而CTE在保存上下文的时候却要保存更多的信息. 尝试对比它们, 并思考两者保存信息不同是什么原因造成的.

异常处理比函数调用更复杂一些：函数调用能保证 callee 的操作满足 calling convention，但中断之后会涉及更复杂的操作，比如特权级切换等，这个过程中每个寄存器的值都可能会被改写。为了保证恢复上下文时程序的状态机不会变，我们需要保存更多的信息。

诡异的 x86 代码

> x86的`trap.S`中有一行`pushl %esp`的代码, 乍看之下其行为十分诡异. 你能结合前后的代码理解它的行为吗? Hint: 程序是个状态机.

这句话和 riscv 的 `trap.S` 中的 `mv a0, sp` 没有本质区别，都是在为即将执行的 `__am_irq_handle` 准备参数。这里的 esp 的内容就是上下文结构体的地址。

从加4操作看CISC和RISC

> 事实上, 自陷只是其中一种异常类型. 有一种故障类异常, 它们返回的PC和触发异常的PC是同一个, 例如缺页异常, 在系统将故障排除后, 将会重新执行相同的指令进行重试, 因此异常返回的PC无需加4. 所以根据异常类型的不同, 有时候需要加4, 有时候则不需要加.
>
> 这时候, 我们就可以考虑这样的一个问题了: 决定要不要加4的, 是硬件还是软件呢? CISC和RISC的做法正好相反, CISC都交给硬件来做, 而RISC则交给软件来做. 思考一下, 这两种方案各有什么取舍? 你认为哪种更合理呢? 为什么?

我认为从抽象层的角度来看由软件处理是否+4更加合理，因为是否需要+4取决于中断的类型，而判断中断类型的是操作系统。因此软件判断中断类型后由软件决定这次中断是否需要再次执行。

mips32延迟槽和异常

> 我们在PA2中提到, 标准的mips32处理器采用了分支延迟槽技术. 思考一下, 如果标准的mips32处理器在执行延迟槽指令的时候触发了异常, 从异常返回之后可能会造成什么问题? 该如何解决? 尝试RTFM对比你的解决方案.

mips32引入了分支延迟槽，所以在遇到跳转指令且确实发生跳转时的指令执行流为：

```
跳转指令 --> 分支延迟槽 --> 跳转的目标指令
```

如果在分支延迟槽指令处出现了异常陷入内核态处理，处理结束之后我们不能按照惯例回到分支延迟槽的 snpc 执行。因为在流水线 CPU 中，经过了异常处理程序之后，跳转指令早就不在 CPU 中了，如果进入 snpc 则无法跳转。所以我们应该将 pc 拨回到跳转指令，将跳转指令再次送入流水线，然后按照顺序再做一次分支延迟槽，这一次分支延迟槽不一定再会触发异常（比如前一次已经处理了缺页），然后就能正确跳转到目标指令继续执行。

补充

关于 `ecall` 指令

ecall 指令没有任何参数，那么如何直到应该向 mcause 寄存器里写入哪种中断原因呢？RTFSC 或者查看 riscv-linux 的约定可以看到 ecall 指令利用 `a7` 寄存器来保存调用号。因此将 `a7` 的内容写入 mcause 即可。

关于恢复上下文中的软件 +4

硬件恢复上下文的方法是在 `mret` 指令中将 dnpc 设置为 mpec+4，但按照讲义中所写，riscv 应该用软件恢复上下文。笔者刚开始尝试了写入一段内联汇编的代码，利用 `csrr` `csrw` 指令直接修改 mepc 的值，但失败了。经过分析发现处理完中断之后 trap.S 会根据上下文结构体 c 的内容来恢复上下文，所以即使中间修改了 mepc 的值，最后仍然会被恢复为没有 +4 的原始值。因此软件+4的方法非常简单：在 do_event 中的 EVENT_YIELD 部分，将 c->mepc 直接 +4 即可。

用户程序和系统调用

必答题

hello程序是什么, 它从何而来, 要到哪里去

> 我们知道`navy-apps/tests/hello/hello.c`只是一个C源文件, 它会被编译链接成一个ELF文件. 那么, hello程序一开始在哪里? 它是怎么出现内存中的? 为什么会出现在目前的内存位置? 它的第一条指令在哪里? 究竟是怎么执行到它的第一条指令的? hello程序在不断地打印字符串, 每一个字符又是经历了什么才会最终出现在终端上?

由于此时还没有实现文件系统，所以 `ramdisk.img` 中只有 hello 程序。nanos-lite 中的内联汇编程序 `resources.S` 将 `ramdisk.img` 的内容加载到了内存中。`resources.S` 的内容如下：

```
.section .data
.global ramdisk_start, ramdisk_end
ramdisk_start:
.incbin "build/ramdisk.img"
ramdisk_end:

.section .rodata
.globl logo
logo:
.incbin "resources/logo.txt"
.byte 0
```

一个有意思的细节是 `resources.S` 中定义了全局变量 `ramdisk_start` 和 `ramdisk_end`，这样在 C 程序中我们可以很方便地定位 `ramdisk.img` 被加载到了内存中的哪个位置。从硬件层面来说，由于现在还没有引入虚拟内存，`ramdisk.img` 被放在了地址 0x83000000 处，Makefile 的 LNK_ADDR 实现了这一点。

在调用了 `naive_uload` 函数后，系统会进入 `hello` 程序的汇编代码（elf文件中有入口地址），hello 程序输出字符的时候，会使用 `write` 系统调用（直接使用 write 或者通过 printf 库函数使用 write），nanos-lite 中的 `fs_write` 函数会使用 AM 提供的接口来输出（在实现了文件系统之后，`fs_write` 会直接使用 `stdout` 文件对应的写函数 `serial_write`）。

思考题

堆和栈在哪里？

> 我们提到了代码和数据都在可执行文件里面, 但却没有提到堆(heap)和栈(stack). 为什么堆和栈的内容没有放入可执行文件里面? 那程序运行时刻用到的堆和栈又是怎么来的? AM的代码是否能给你带来一些启发?

AM 的作用就是在硬件层的基础上进行了一定程度的抽象，提供了运行时环境。堆和栈是一种必要的运行时环境（事实上就是 nemu 内存大数组的一部分），因此可执行文件中只需要刻画如何使用堆和栈，并不需要把堆和栈放在文件里。

如何识别不同格式的可执行文件？

> 如果你在GNU/Linux下执行一个从Windows拷过来的可执行文件, 将会报告"格式错误". 思考一下, GNU/Linux是如何知道"格式错误"的?

可执行文件的前几个字符是文件的魔数，用于标识文件的类型。GNU/Linux 在执行文件之前可以先检查魔数来判断这是否是一个 ELF 格式的可执行文件。

冗余的属性？

> 使用`readelf`查看一个ELF文件的信息, 你会看到一个segment包含两个大小的属性, 分别是`FileSiz`和`MemSiz`, 这是为什么? 再仔细观察一下, 你会发现`FileSiz`通常不会大于相应的`MemSiz`, 这又是为什么?

该问题的解答见下一道思考题“为什么要清零”。

为什么要清零？

> 为什么需要将`[VirtAddr + FileSiz, VirtAddr + MemSiz)`对应的物理区间清零?

SFTW 后发现这是一个非常有意思的问题。ELF 文件中的 Filesize 指的是这个段在 ELF 文件中占据的大小，而 Memsize 指的是这个段最终在内存中占据的大小。这两者之所以不一样是因为 .bss 节（未初始化变量）在 ELF 中是没有分配具体的空间的，只是一个占位符，但最终在运行状态下它们也需要被分配内存空间。

将`[VirtAddr + FileSiz, VirtAddr + MemSiz]`对应的物理区间清零，相当于使得 .bss 中的变量的初值都为 0。我们在 C 语言程序中定义未初始化的全局变量，其默认初值为0，正是因为加载器在加载时将这一段内存置为了0。经过笔者的实验，如果将这一清空语句删除，运行程序将会触发段错误。[这篇回答](#)对这个问题给出了较为详细的解释。

系统调用的必要性

> 对于批处理系统来说, 系统调用是必须的吗? 如果直接把AM的API暴露给批处理系统中的程序, 会不会有问题呢?

AM 的 API 只能负责完成相应的功能，而系统调用应该还具有统一调度各个程序对硬件的使用的功能。比如系统调用 read，为了安全性的考虑，它应该只能读取本用户程序相关的内存区域，不能访问别的用户程序的内存。这部分安全检查就应该由系统调用及其背后的操作系统来完成。AM 的 API 只是硬件功能的简单封装，无法实现这样的功能。

RISC-V系统调用号的传递

> 如果你选择的是RISC-V, 你会发现它并不是通过`a0`来传递系统调用号. 事实上, 我们参考了RISC-V Linux的系统调用参数传递的约定: 即在RISC-V Linux上也是通过这个寄存器来传递系统调用号的. 你觉得RISC-V Linux为什么没有使用`a0`来传递系统调用号呢?

RISC-V Linux 调用约定中使用 `a7` 来传递系统调用号，且规定系统调用最多有 6 个参数，存放在 `a0~a5` 中。我认为这样设计可能的目的是保持系统调用和函数调用在传递参数上的一致性。

补充

检查 ELF 文件的魔数

> 我们知道ELF文件的开头都有一个特殊的魔数, 为了防止loader加载了一个非ELF格式的文件, 我们可以在loader中对魔数进行检查:
>
> ```
> assert(*(uint32_t *)elf->e_ident == 0xBadC0de);
> ```
>
> 你需要把上述的`0xBadC0de`换成正确的魔数.
>
> 别小看这个表面上很蠢的`assert()`, 当你哪天手抖不知道做了什么而又被它抓到的时候, 你就知道谢天谢地了.

ELF文件开头的四个字节是 `7f 45 4c 46` ，注意到 riscv 是小端机，所以正确的魔数为 0x464c457f。

检测 ELF 文件的ISA类型

> 你很有可能会因为疏忽, 从而让`native`的Nanos-lite来加载运行一个x86/mips32/riscv32的dummy. 从ISA规范的角度来说, 这种行为显然属于UB, 具体而言通常会发生一些难以理解的错误. 为了避免这种情况, 你可以在loader中检测ELF文件的ISA类型. 我们可以根据AM中定义的一些宏来筛选出预期的ISA类型，然后和ELF信息中的某个域进行对比, 如果发现要加载的ELF文件的ISA类型和预期不一致, 就报错. 如果你不知道AM中的宏在哪里定义, RTFSC. 如果你不知道应该和ELF中的哪个域进行对比, RTFM.

当前可能需要用到的 ISA 应该只有 native 的 x86-64 和 nemu 的 riscv32，因此笔者使用了如下代码来定义宏 `EXPECT_ISA`：

```
#if defined(__ISA_AM_NATIVE)# define EXPECT_TYPE EM_X86_64#elif defined(__ISA_RISCV32__)# define EXPECT_TYPE EM_RISCV   // 阅读 elf.h 中的宏可以找到 RISC-V 对应 EM_RISCV#else# error Unsupported ISA#endif
```

ELF 文件中的 ISA 属性存放在 elf_header 的 e_machine 字段中。因此用一个 assert 语句进行比较即可。

关于内存的读写

笔者曾经想了很久也不明白应该如何读写内存（因为在 AM 的 nemu 相关文件夹中没有找到 API）。后来想起来内存的读写作为一条架构无关的功能，放在了 klib 里面。因此直接调用 memset, memcpy 等即可。从 ELF 文件中读出来的 vaddr 虽然是变量，但里面存储的就是地址，直接将其转换成指针类型即可。

关于跳转

编译器编译并生成可执行文件时，会寻找 `start.S` 并将其作为程序的入口。这个入口保存在了 elf header 的 e_entry 字段中，因此直接访问该字段即可。框架代码中有一个很好的细节：

```
((void *)())entry) ();
```

它将保存有客户程序入口地址的变量 entry 强制转换成了一个函数指针。然后调用这个函数，相当于跳转到函数指针指向的地址，而这个地址就是 entry 原来存储的值。这一句代码用 C 的方式完成了汇编级别的跳转。

关于返回值

> 为什么我使用了框架代码给定的宏 GPRx 来存储返回值，但堆区的实现还是失败了？

检查框架代码可以发现，除了 `GPR1` 对应到了正确的 `a7` 以外，其他的都是错的！根据 riscv-linux 的 syscall 约定，我们应该将 `GPR2~GPR4` 对应到 `a0~a2`，返回寄存器 `GPRx` 对应到 `a0`，这样就可以正确地提供返回值了。

文件系统

补充

关于 `file-test` 中文件读入的溢出问题

笔者起初认为进行读取操作时，读取的范围是不能超过本文件大小的，因此在读取的时候对 offset 和 len 进行了 assert，结果触发了 fail。

笔者打印了每次读取的 len，发现 fscanf 是**批量读取**的：由于有一次读取在四千多的位置一下读取了 1024 个字符，所以触发了断言。

笔者再次阅读了 read 系统调用的手册，发现其返回值理应是实际读取到的字符个数。处于安全性的考虑，一个应用程序不应该可以读取到别的用户程序的数据信息，所以如果在文件系统中剩余的数据不足给定的 len，则应该只将剩余的数据放入缓冲区，并返回真实的读取到的字符数。

精彩纷呈的应用程序

思考题

比较 fixedpt 和 float

> `fixedpt`和`float`类型的数据都是32位, 它们都可以表示2^32个不同的数. 但由于表示方法不一样, `fixedpt`和`float`能表示的数集是不一样的. 思考一下, 我们用`fixedpt`来模拟表示`float`, 这其中隐含着哪些取舍?

fixedpt 中每相邻的两个数之间的差都是 2^{-8}，而 float 在越小的数据范围下表示的精度越高。通常来说我们在表示大数的时候会对其小数点后的精度要求不那么高，也就是我们更在意数据的“相对误差”而不是“绝对误差”，在这一点上 float 会更加合理。

神奇的fixedpt_rconst

> 阅读`fixedpt_rconst()`的代码, 从表面上看, 它带有非常明显的浮点操作, 但从编译结果来看却没有任何浮点指令. 你知道其中的原因吗?

笔者暂时没有研究出来，留坑。

神奇的LD_PRELOAD

> `bmp-test`需要打开一个路径为`/share/pictures/projectn.bmp`的文件, 但在Linux native中, 这个路径对应的文件并不存在. 但我们还是把`bmp-test`成功运行起来了, 你知道这是如何实现的吗? 如果你感兴趣, 可以在互联网上搜索`LD_PRELOAD`相关的内容.

`LD_PRELOAD` 可以实现路径的重定向，PA4.1 的 busybox 也使用了相似的技术实现了将 shell 命令的地址重定向到 busybox 中的程序。具体内容尚未了解，留坑。

仙剑奇侠传是如何工作的？

> 我们在PA2中讨论过一个游戏的基本框架, 尝试阅读仙剑奇侠传的代码, 找出基本框架是通过哪些函数实现的. 找到之后, 可能会对你调试仙剑奇侠传带来一定的帮助. 虽然仙剑奇侠传的代码很多, 但为了回答这个问题, 你并不需要阅读大量的代码.

PA2 中提到的一个游戏的基本框架大致为：

```
while (1) {  等待新的一帧();  // AM_TIMER_UPTIME  处理用户按键();  // AM_INPUT_KEYBRD  更新游戏逻辑();  // TRM  绘制新的屏幕();  // AM_GPU_FBDRAW}
```

仙剑奇侠传中的 `PAL_GameMain()` 中的 while 循环大抵做了相似的工作：

```
while (TRUE)    {       //      // Do some initialization at game start.      //
     if (gpGlobals->fGameStart)      {          PAL_GameStart();
gpGlobals->fGameStart = FALSE;      }      //      // Load the game resources
if needed.      //      PAL_LoadResources();      //      // Clear the input
state of previous frame.      //      PAL_ClearKeyState();      //      // Wait
for the time of one frame. Accept input here.      //
PAL_DelayUntil(dwTime);      uint32_t now = SDL_GetTicks();
UpdateFPS(now);      //      // Set the time of the next frame.      //
dwTime = now + FRAME_TIME;      //      // Run the main frame routine.      //
     PAL_StartFrame();    }
```

`PAL_LoadResources()` 用于加载需要的游戏资源。`PAL_ClearKeyState()` 清空之后，`PAL_DelayUntil()` 会持续执行一段时间并接收设备的输入（主要使用了 `SDL_Delay()` 和 `PAL_ProcessEvent()`）。`PAL_StartFrame()` 用于更新游戏逻辑（比如根据输入来改变人物的位置）并绘制图像。

仙剑奇侠传的脚本引擎

> 在`navy-apps/apps/pal/repo/src/game/script.c`中有一个`PAL_InterpretInstruction()`的函数, 尝试大致了解这个函数的作用和行为. 然后大胆猜测一下, 仙剑奇侠传的开发者是如何开发这款游戏的? 你对"游戏引擎"是否有新的认识?

这个函数的行为有一点像一个 cpu：它拿到“指令”之后，会进行译码，执行，函数会返回下一条应该执行的“指令”的地址。

仙剑的开发者在开发这款游戏的时候，可能发明了一款专用于 pal 的 ISA，然后所有和游戏相关的逻辑更新的操作都通过这个 ISA 支持的指令来完成。 游戏引擎其实就是一个虚拟机。如果我们认为 nemu 是真机的话，那么 pal 里的这个专门执行 pal-ISA 指令集的 "cpu" 就是游戏引擎；如果我们认为 native 是真机的话，笔者目前的 nemu 就是一个执行 riscv32-ISA 指令集的“游戏引擎”。

不再神秘的秘技

> 网上流传着一些关于仙剑奇侠传的秘技, 其中的若干条秘技如下:
>
> 1. 很多人到了云姨那里都会去拿三次钱, 其实拿一次就会让钱箱爆满! 你拿了一次钱就去买剑把钱用到只剩一千多, 然后去道士那里, 先不要上楼, 去掌柜那里买酒, 多买几次你就会发现钱用不完了.
> 2. 不断使用乾坤一掷(钱必须多于五千文)用到财产低于五千文, 钱会暴增到上限, 如此一来就有用不完的钱了
> 3. 当李逍遥等级到达99级时, 用5~10只金蚕王, 经验点又跑出来了, 而且升级所需经验会变回初期5~10级内的经验值, 然后去打敌人或用金蚕王升级, 可以学到灵儿的法术(从五气朝元开始); 升到199级后再用5~10只金蚕王, 经验点再跑出来, 所需升级经验也是很低, 可以学到月如的法术(从一阳指开始); 到299级后再用10~30只金蚕王, 经验点出来后继续升级, 可学到阿奴的法术(从万蚁蚀象开始).
>
> 假设这些上述这些秘技并非游戏制作人员的本意, 请尝试解释这些秘技为什么能生效.

笔者没有仔细玩过仙剑奇侠传，因此对其中的一些游戏机制不太了解。但是从上述的某些 bug 的现象中（比如第二个）可以推测有一些无符号数在下溢出之后直接变成了 UINT_MAX，从而导致问题。

在 Navy 中运行 microbench

> 尝试把microbench编译到Navy并运行, 你应该会发现运行错误, 请尝试分析原因.

笔者暂时还未实现 Navy 中的 AM，留坑。

如何在 Navy 上运行 Nanos-lite?

> 既然能在Navy上运行基于AM的FCEUX, 那么为了炫耀, 在Navy上运行Nanos-lite也并不是不可能的. 思考一下, 如果想在Navy上实现CTE, 我们还需要些什么呢?

我们需要支持 CTE 的中断机制。在 Nanos-lite 中有一些汇编操控特权寄存器的语句，这些语句直接与硬件交互。但现在 Navy-Nanos-lite 是在 Nanos-lite 上跑的，所以特权寄存器的一套机制需要有一种软的处理方案。

RTFSC???

> 机智的你也许会想: 哇塞, 下学期的oslab0我不就有优秀代码可以参考了吗? 不过我们已经对发布的代码进行了某种特殊的处理. 在沮丧之余, 不妨思考一下, 如果要你来实现这一特殊的处理, 你会如何实现? 这和PA1中的表达式求值有什么相似之处吗?

观察 oslab0 的代码，我们基本可以确定代码的发布者对代码中的变量名，函数名等进行了替换，将其换成了冗长的不可读字符串。PA1 中用正则表达式匹配字符串的技术在这里可以派上用场。

终极拷问

> 自古以来, 计算机系统方向的课程就有一个终极拷问:
>
> > 当你在终端键入`./hello`运行Hello World程序的时候, 计算机究竟做了些什么?
>
> 你已经实现了批处理系统, 并且成功通过NTerm来运行其它程序. 尽管我们的批处理系统经过了诸多简化, 但还是保留了计算机发展史的精髓. 实现了批处理系统之后, 你对上述的终极拷问有什么新的认识?

一个简单的流程如下：

- 键入 `./hello` 以后，shell 解析命令之后发现要运行 hello 文件（可执行程序），于是使用 `execve` 系统调用，将文件名 hello 和相应的参数，环境变量传进去。
- 陷入内核态之后，操作系统检查 hello 文件是否存在，是否有足够的访问权限等，如果有问题会返回相应的错误号。shell 如果发现 `execve` 系统调用返回，则会根据错误号报错。
- 如果一切正常，`execve` 系统调用不会返回。操作系统会为 hello 程序创建一个初始的上下文结构体，然后调度 hello 作为下一个执行的进程。（在 pa3 中暂时没有进程的概念，会直接使用 naive_uload 来加载 hello 并跳转到 entry 执行）。
- hello 程序使用 printf 库函数打印字符串 "Hello, world!\n"，printf 库函数的底层会使用 `write` 系统调用，陷入内核态之后，操作系统使用 `write` 系统调用相关的服务程序，向 stdout (屏幕) 输出字符。

补充

关于 `convert.sh` 的报错

直接运行 `convert.sh` 会报错

```
attempt to perform an operation not allowed by the security policy `PDF'
```

STFW后得知，只需将 `/etc/ImageMagick-6/policy.xml` 中的

```
<policy domain="coder" rights="none" pattern="PDF" />
```

中的 `"none"` 改为 `"read | write"` 即可。

关于仙剑奇侠传的红蓝颜色

笔者的仙剑奇侠传刚开始启动页面出现的是红色的旗子，游戏中的所有人物都是青色的脸庞。仔细阅读 SDL 部分的源码后发现，SDL 中对于 32 位颜色数据的解读和讲义中写的略有不同：正确的解读方法应该是 `aabbggrr`，和讲义中的相比正好红色和蓝色颠倒了。

关于仙剑奇侠传的字幕黑框

笔者的仙剑奇侠传曾经出现所有有字幕的地方（比如人物的对话区域，以及“新的故事”“旧的回忆”的选择区域）全部是黑框的现象。仔细阅读 SDL 手册后发现 `SDL_UpdateRect()` 函数实现的有一些问题，导致一直在取屏幕左上角的一小块矩形来更新像素，那一部分恰好是黑色的。修正矩形区域的坐标后问题解决。

仙剑奇侠传与 `SDL_FillRect()`

`SDL_FillRect()` 的官方文档的描述是将画布中指定矩形区域的颜色都置为 color。其中的参数 color 是一个 32 位数。但仙剑奇侠传中是使用调色盘来表示颜色的，pixels 里存放的是颜色在调色盘中的索引，应该怎么将这个 color 赋过去呢？

笔者起初以为仙剑中调用该函数时传入的 color 应该是 32 位的 `aaggbbrr` 格式的颜色，因此将调色盘中的所有颜色取出和 color 一一比对，但发现 color 并不一定在调色盘中。笔者尝试使用曼哈顿距离最小的颜色来代替 color，但效果很差。笔者又尝试了从调色盘中随机挑一个颜色，将 color 替换进去。这样虽然可以达到正确的效果，但存在一定的隐患。

笔者在阅读了 pal 的源码之后，发现其中调用 `SDL_FillRect()` 的时候传递的 color 参数都是 8 位二进制数，明白其实传进去的就是颜色的调色盘索引，所以直接赋值即可。

PA4 - 虚实交错的魔法: 分时多任务

实验进度

我完成了所有的必答题，并针对选做题和思考题给出了自己的想法。

必答题

分时多任务的具体过程

> 请结合代码, 解释分页机制和硬件中断是如何支撑仙剑奇侠传和hello程序在我们的计算机系统(Nanos-lite, AM, NEMU)中分时运行的.

这里主要叙述从 hello 用户程序切换到 pal 的过程。在抢占式多任务系统中，hello 相当于一个“恶意用户程序”，因为它不会主动使用 `yield()` 来让操作系统切换程序。但实现了时钟中断之后我们可以强制 hello 的挂起。

每一条指令执行结束时，nemu 的 `cpu_exec()` 中都会调用 `timer_interrupt()` 函数来检查 CPU 的 INTR 引脚有没有被拉高。如果被拉高了说明有设备发出了中断请求（在 PA 中只有时钟中断请求），此时 CPU 会将 mstatus 寄存器的 MIE 位置零（防止中断嵌套），然后进入 nanos-lite 的中断/异常处理程序 `trap.S`。

在 `trap.S` 中，nanos-lite 首先会将 sp 指针切换到 hello 用户进程的内核栈，在内核栈上保存 hello 进程的上下文，将 mscratch 和 c->np 等信息设置好之后，进入 `__am_irq_handle()` 函数。

在 `__am_irq_handle()` 函数中，nanos-lite 发现 mcause 寄存器中的事件编号是 0x80000007，于是标记该事件为时钟中断，调用 `do_event()` 函数。`do_event()` 函数中调用了 `shedule()` 函数，`schedule()` 函数将 current->cp 指向刚才保存在 hello 内核栈上的 hello 上下文，将 current 指针指向 pal 进程的 pcb，并返回 pal 进程内核栈上的上下文结构体（该结构体可能是 `context_uload()` 时创建的初始结构体，也可能是前一次进程切换时保存的结构体）。`__am_irq_handle()` 函数最终会返回 pal 进程的上下文结构体。

回到 `trap.S` 中以后，nanos-lite 会恢复 pal 进程的上下文，并且根据 c->np 将栈指针从 pal 进程的内核栈移到 pal 进程的用户栈。执行完 `mret` 指令之后，nanos-lite 就切换到了 pal 的代码继续执行（`mret` 指令会将 mstatus 的 MIE 位重新打开，以接收中断）。

理解计算机系统

> 尝试在Linux中编写并运行以下程序:
>
> ```
> int main() {    char *p = "abc";    p[0] = 'A'; return 0;}
> ```
>
> 你会看到程序因为往只读字符串进行写入而触发了段错误. 请你根据学习的知识和工具, 从程序, 编译器, 链接器, 运行时环境, 操作系统和硬件等视角分析"字符串的写保护机制是如何实现的". 换句话说, 上述程序在执行`p[0] = 'A'`的时候, 计算机系统究竟发生了什么而引发段错误? 计算机系统又是如何保证段错误会发生? 如何使用合适的工具来证明你的想法?

字符串 "abc" 在编译的时候会被放入 ELF 文件的只读数据段，并在链接运行的时候位于只读代码段。高级语言中的 `p[0] = 'A'` 实际上是一次访存写入操作，对应的汇编代码为

```
0000000000000000 <main>:   0:   f3 0f 1e fa             endbr64    4:   55
                 push   %rbp   5:    48 89 e5                mov    %rsp,%rbp
8:   48 8d 05 00 00 00 00    lea    0x0(%rip),%rax        # f <main+0xf>   f:
48 89 45 f8             mov    %rax,-0x8(%rbp)  13: 48 8b 45 f8             mov
   -0x8(%rbp),%rax  17: c6 00 41                movb   $0x41,(%rax)          #
p[0] = 'A'  1a:  b8 00 00 00 00          mov    $0x0,%eax  1f:   5d
         pop    %rbp  20:    c3                      ret
```

执行到 <main+0x17> 这条指令时，硬件中的 MMU 会检查当前进程的权限是否可以访问这一内存区域。由于 "abc" 是只读数据，所以发生了越权行为。访问非法地址后 MMU 会修改一些特权寄存器的值记录信息，然后转到操作系统的异常处理程序。操作系统会向用户进程发送一个 SIGSEGV 信号，表示发生了段错误。用户进程收到 SIGSEGV 信号后的默认动作是终止程序运行，但它会生成一个核心文件以帮助调试，也就是所谓的 (core dumped 核心已转储)。

我们可以在用户程序中注册一个信号处理函数来捕捉 SIGSEGV 信号，代码如下：

```
#include <stdio.h>#include <signal.h>#include <stdlib.h>void signal_handler(int signum){    if (signum == 11) printf("Signal SIGSEGV captured\n");
exit(0);}int main() {    signal(SIGSEGV, signal_handler); char *p = "abc";
p[0] = 'A'; return 0;}
```

编译、运行该程序后不会报段错误，而会打印 Signal SIGSEGV captured 之后正常退出。这可以证明用户进程确实接收到了 SIGSEGV 信号。

我们也可以通过 gdb 来检测 SIGSEGV 信号：在 gdb 中使用 `layout asm` 打开汇编代码，一条一条执行，在执行到 `movb` 语句时会显示 SIGSEGV 信号。

为了看到操作系统执行该程序时经历的过程，我们可以用 strace 工具来查看执行 `./a.out` 时使用的所有系统调用。（注意：strace 工具默认将信息输出到标准错误流，所以如果想用 less 工具来查看的话，需要先将输出重定向到标准输出流，使用命令 `strace ./a.out 2>&1 | less`。）

```
execve("./a.out", ["./a.out"], 0x7ffec38443f0 /* 67 vars */) = 0brk(NULL)
                        = 0x555ca9bfc000arch_prctl(0x3001 /* ARCH_??? */,
0x7ffd43e30600) = -1 EINVAL (Invalid argument)access("/etc/ld.so.preload",
R_OK)      = -1 ENOENT (No such file or directory)openat(AT_FDCWD,
"/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3newfstatat(3, "",
{st_mode=S_IFREG|0644, st_size=76978, ...}, AT_EMPTY_PATH) = 0mmap(NULL, 76978,
PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fe701222000close(3)
       = 0openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6",
O_RDONLY|O_CLOEXEC) = 3read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\240\206\2\0\0\0\0\0"..., 832) =
832pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) =
784pread64(3, "\4\0\0\0
\0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0\0"..., 48, 848) =
48pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\0+H)\227\201T\214\233\304R\352\306\3379\220%"...,
68, 896) = 68newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=1983576, ...},
AT_EMPTY_PATH) = 0mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe701220000pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) =
784mmap(NULL, 2012056, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7fe701034000mmap(0x7fe70105a000, 1486848, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x26000) =
0x7fe70105a000mmap(0x7fe7011c5000, 311296, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x191000) =
0x7fe7011c5000mmap(0x7fe701211000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1dc000) =
0x7fe701211000mmap(0x7fe701217000, 33688, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fe701217000close(3)
                     = 0mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe701032000arch_prctl(ARCH_SET_FS,
0x7fe701221580) = 0mprotect(0x7fe701211000, 12288, PROT_READ) =
0mprotect(0x555ca947c000, 4096, PROT_READ) = 0mprotect(0x7fe701267000, 8192,
PROT_READ) = 0munmap(0x7fe701222000, 76978)           = 0rt_sigaction(SIGHUP,
{sa_handler=0x555ca947a189, sa_mask=[HUP], sa_flags=SA_RESTORER|SA_RESTART,
sa_restorer=0x7fe701075040}, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) =
0--- SIGSEGV {si_signo=SIGSEGV, si_code=SEGV_ACCERR, si_addr=0x555ca947b01c} --
-+++ killed by SIGSEGV (core dumped) +++Segmentation fault (core dumped)
```

虽然笔者不能看懂所有的系统调用，但大概可以看到一些比较核心的步骤，比如使用 `execve` 系统调用转到 `a.out` 程序执行，中间的 `mmap` 系统调用试图访问了一些有着 `MAP_DENYWRITE` 属性的东西。最后发出了 SIGSEGV 信号，并给出了触发该信号的指令地址。

以下是我的自由报告内容，我在做实验的过程中详细记录了自己遇到的所有问题和查阅的所有资料。这些笔记按照章节排列，每章通常有两个部分

- 思考题：包含笔者针对思考题查阅的资料和我尝试给出的答案。
- 补充：这里的内容是没有在讲义中提及的问题，以及讲义中建议自学的东西。

多道程序

思考题

为什么需要使用不同的栈空间?

> 如果不同的进程共享同一个栈空间, 会发生什么呢?

比如不同的进程可以互相修改别人的栈的内容，这样原先的进程就可能崩溃。

mips32 和 riscv32 的调用约定

> 我们没有给出mips32和riscv32的调用约定, 你需要查阅相应的ABI手册. 当然, 你也可以自己动手实践来总结传参的规则.

笔者选择的架构是 riscv32，所以在此简单地列举 riscv32 的调用约定：riscv32 使用寄存器来传递参数和返回值。`a0` `a1` `a2` 等寄存器分别存储了第一，第二，第三个参数，以此类推；`a0` 寄存器存储了返回值。

一山不能藏二虎?

> 尝试把`hello_fun()`换成Navy中的`hello`:
>
> ```
> -context_kload(&pcb[0], (void *)hello_fun, NULL);+context_uload(&pcb[0], "/bin/hello");context_uload(&pcb[1], "/bin/pal");
> ```
>
> 你发现了什么问题? 为什么会这样? 思考一下, 答案会在下一阶段揭晓!

程序触发了段错误。两个用户程序使用的是相同的用户栈，代码也会加载到同样的位置，它们会相互覆盖，从而导致错误。

为什么少了一个const?

> 在`main()`函数中, `argv`和`envp`的类型是`char * []`, 而在`execve()`函数中, 它们的类型则是`char *const []`. 从这一差异来看, `main()`函数中`argv`和`envp`所指向的字符串是可写的, 你知道为什么会这样吗?

根据 C99 手册中的规定：

> **Program Startup**
>
> The parameters `argc` and `argv` and the strings pointed to by the `argv` array shall be modifiable by the program, and retain their last-stored values between program startup and program termination.

STFW 得出的结论是，这是一个历史遗留问题。早期的 C 语言没有 const，后来 C++ 为了兼容之前的 C 程序，也没有加上这个 const。有一些程序正是利用 `argv[]` 的可修改性进行工作的，例如：

```
// print out all the arguments:while (--argc)    std::cout << *++argv << std::endl;
```

此外，一些 UNIX API，比如 `getopt`，确实会修改 `argv[]` 的内容。但 `argv[]` 作为一个栈上的局部变量，在定义的时候加上 const 也是无伤大雅的。（stackoverflow 上的这个回答比较详细）

补充

调试心得

字符串的复制最好使用 `strcpy` 函数，它会帮你将字符串末尾的 `\0` 也一并复制过去。如果想要使用 `memcpy` 来复制的话，长度一定要设置成 `strlen(src)+1` ，否则会漏复制末尾的 `\0` 从而导致奇怪的后果。

关于用户程序参数的错误

笔者在实现了 nterm 的内建 shell 的参数传送功能之后，在使用命令 `pal --skip` 来跳过仙剑奇侠传的动画时发现无效。简单排查之后发现存储参数字符串的内存在一个奇特的时刻被改写了。仔细排查之后发现，这些参数字符串是被仙剑奇侠传用户程序改写的——这其中有一些机缘巧合：笔者在 nterm 中定义了一个数组来存放参数字符串，因此参数字符串的内容作为客户程序的一部分，在 nterm 运行时会被加载到 `0x83000000` 附近。笔者刚开始实现的 loader 函数在切换用户程序时，会先加载新的用户程序，再在用户栈中安放用户程序的参数，这导致 pal 程序在加载到 `0x83000000` 附近时直接覆盖了 nterm 中还没来得及拿出来的参数字符串。因此将这两个步骤换个顺序——先把参数字符串放到用户栈里，再加载程序即可。

这个问题的发生归根结底在于不同的用户程序使用了同一段物理内存来进行工作。在实现了虚拟内存之后应该就不会有这个问题。

关于添加 busybox 之后的错误

笔者在添加了busybox 用户程序之后，nanos-lite 直接崩溃。检查堆区的范围后发现，`heap.start` 此时已经到了 `0x83000000` 之后，因此加载用户程序时用户程序的代码会直接覆盖用户栈的内容。

> 这说明 ramdisk.img 的大小的上限并不是 48MB，而是更小的一个阈值。因为 heap.start 考虑到一些对齐原因并不是紧贴着 ramdisk_end 设置的。

笔者在 nslider 用户程序中删掉了几张图片，将 ramdisk.img 的大小降到 37MB 之后，恢复正常。

虚实交错的魔法

思考题

[二周目] 实现基于PIE的loader

> 天下并没有免费的午餐, PIE之所以能做到位置无关, 其实是要依赖于程序中一个叫GOT(global offset table, 全局偏移量表)的数据结构. 要正确运行PIE, 加载器需要在加载程序的时候往GOT中填写正确的内容.
>
> 有兴趣的同学可以让Nanos-lite的loader支持PIE, 当然这需要了解一些ELF相关的细节, 具体细节可以参考ABI手册.

留坑，二周目再做。

笔者虽然还没有实现基于 PIE 的 loader，但学习了一下动态加载的基本原理。以下面的程序为例：

```
extern int foo;int func(void){    return foo;}
```

在 amd64 架构下，访问变量 foo 的核心语句是

```
mov 0x200271(%rip), %rax    # 200828
```

200828 处是 GOT 的一个表项：

```
Offset          Info           Type           Sym. Value    Sym. Name +
Addend[...]000000200828  000400000006 R_X86_64_GLOB_DAT 0000000000000000 foo +
0
```

动态加载器在加载动态链接库时，会往 GOT 中填写相关的表项。

在 i386 架构中没有类似 %rip 这样的寄存器，无法直接得知当前指令的位置，因此通常会使用类似于下面这样的略带 tricky 的代码：

```
0000040c <function>: 40c:    55          push   %ebp 40d:    89 e5
   mov    %esp,%ebp 40f:    e8 0e 00 00 00          call   422
<__i686.get_pc_thunk.cx> 414:    81 c1 5c 11 00 00       add    $0x115c,%ecx
41a:    8b 81 18 00 00 00       mov    0x18(%ecx),%eax 420:    5d
      pop    %ebp 421:    c3                      ret00000422
<__i686.get_pc_thunk.cx>: 422:    8b 0c 24        mov    (%esp),%ecx 425:    c3
                    ret
```

在 40f 处调用了一个函数 `__i686.get_pc_thunk.cx>`，这个函数将 call 的返回地址（也就是 414）保存到了 %ecx 寄存器中，从而变向实现了 get_pc 的功能。

超越容量的界限

思考题

虚存管理中PIC的好处

> 我们之前提到, PIC的其中一个好处是可以将代码加载到任意内存位置执行. 如果配合虚存管理, PIC还有什么新的好处呢? (Hint: 动态库已经在享受这些好处了)

PIC 的好处在于，我们不需要将 PIC 反复装进内存。比如第一个进程将动态库放进了内存，那么第二个进程如果也要用动态库，它只要找一下内存里有没有动态库的页面，如果有就将自己的页表中关于动态库的那项直接指向已有的动态库页面即可。

理解分页细节

> - i386不是一个32位的处理器吗, 为什么表项中的基地址信息只有20位, 而不是32位?
> - 手册上提到表项(包括CR3)中的基地址都是物理地址, 物理地址是必须的吗? 能否使用虚拟地址?
> - 为什么不采用一级页表? 或者说采用一级页表会有什么缺点?

- 每个页的大小是 4KB，所以地址的低 12 位用来表示页内偏移，高 20 位用来指示页号。
- 物理地址是必须的，如果所有的东西都是虚拟地址，那我们就无法找到解析虚拟地址所需的信息。
- 采用一级页表的缺点在于：通常来说页表中的有效项是非常稀疏的。如果采用一级的线性页表，我们不得不将所有的页表内容放进内存，占用相当大的一块内存空间，但其实里面有大量的 0。如果采用二级页表，我们只需要将页目录的一个页和有内容的那些子页放进内存，这样就节省了内存空间。

空指针真的是"空"的吗?

> 程序设计课上老师告诉你, 当一个指针变量的值等于NULL时, 代表空, 不指向任何东西. 仔细想想, 真的是这样吗? 当程序对空指针解引用的时候, 计算机内部具体都做了些什么? 你对空指针的本质有什么新的认识?

严格来说空指针并不是不指向任何东西，而是它指向一个不可以被访问的东西，如果试图对空指针解引用，会因为越权行为触发异常。

mips32的TLB管理是否更简单?

> 有一种观点认为, mips32的分页机制更简单. 你认同吗? 尝试分别在现在, 以及完成这部分内容之后回答这个问题.

mips32 将 TLB 管理完全交给软件，这使得在 PA 中我们不能省略 TLB 的实现，也必须要处理进程切换时产生的一系列一致性问题，在 PA 中完成 mips32 的分页机制更加麻烦。但实际中，硬件上维护 TLB 比软件上维护 TLB 要复杂，需要更多的门电路支持。mips32 的 TLB 管理将硬件的活交给软件，可以使硬件设计更为简单。

内核映射的作用

> 对于x86和riscv32, 在`protect()`中创建地址空间的时候, 有一处代码用于拷贝内核映射:
>
> ```
> // map kernel spacememcpy(updir, kas.ptr, PGSIZE);
> ```
>
> 尝试注释这处代码, 重新编译并运行, 你会看到发生了错误. 请解释为什么会发生这个错误.

我们在从用户进程切换到内核线程的时候并没有切换 satp，因为对于任何一个用户进程来说，内核段都是恒等映射。但这就要求用户进程的页表里必须包含内核映射，不然内核区域的代码无法正确执行。

Native 的 VME 实现

> 尝试阅读`native`的VME实现, 你发现`native`是如何实现VME的? 为什么可以这样做?

留坑。

可以在用户栈里面创建用户进程上下文吗?

> `ucontext()`的行为是在内核栈`kstack`中创建用户进程上下文. 我们是否可以对`ucontext()`的行为进行修改, 让它在用户栈上创建用户进程上下文? 为什么?

目前来说是可以的，事实上除了创建的时候上下文在内核栈上，其他时候上下文都在用户栈上。但在用户栈上创建和保存上下文会遇到下面一道思考题涉及的问题。

并发执行多个用户进程

> 让Nanos-lite加载仙剑奇侠传和hello这两个用户进程; 或者是加载NTerm和hello内核线程, 然后从NTerm启动仙剑奇侠传, 你应该会在运行的时候观察到错误. 尝试分析这一错误的原因, 并总结为了支持这一功能, 我们需要满足什么样的条件.
>
> 这可以说是一周目最难的一道思考题了, 虽然我们会在PA4的最后给出分析, 喜欢挑战的同学仍然可以在这里尝试独立思考: 如果你能独立解决这个问题, 说明你对计算机系统的理解可以说是相当了得了.

以前者为例，当我们想要从 pal 进程切换到 hello 进程的时候，我们首先会把 pal 进程的上下文保存在 pal 的用户栈上，然后陷入内核执行中断服务程序，注意我们在中断服务程序中进行的函数调用，局部变量定义等事情也一直是在 pal 的用户栈上做的。`schedule()` 将下一个进程切换到 hello，做完 `__am_switch()` 后，页表已经是 hello 进程的页表，和 pal 相关的所有虚存映射都不在该页表中。因此目前我们无法访问到保存在 pal 用户栈上的那些和中断服务程序相关的栈帧信息（比如返回地址），所以会发生段错误。

补充

关于 `mm_brk()`

笔者在这里卡了相当长的时间，因为不太能理解讲义的内容。经过思考后大致明白了这个堆区的意思。用户程序申请的新 program-break 都是虚拟内存下的 pb。pcb 结构体中的 max_brk 存的也是虚拟内存下的 pb。如果当前申请的 pb 值超过了 max_brk，就申请一些新的物理内存页，将 [max_brk,pb) 中的内容映射到物理内存页中。想要获得当前进程的 max_brk，只要通过 current 指针访问当前进程的 pcb 即可。

来自外部的声音

思考题

灾难性的后果(这个问题有点难度)

> 假设硬件把中断信息固定保存在某个内存地址(例如`0x1000`)的位置, AM也总是从这里开始构造上下文. 如果发生了中断嵌套, 将会发生什么样的灾难性后果? 这一灾难性的后果将会以什么样的形式表现出来? 如果你觉得毫无头绪, 你可以用纸笔模拟中断处理的过程.

留坑。

如何支持中断嵌套

> 思考一下, x86, mips32和riscv32的软硬件该分别如何协同, 来支持中断嵌套?

留坑。

中断和用户进程初始化

> 我们知道, 用户进程从Navy的`_start`开始运行, 并且在`_start`中设置正确的栈指针. 如果在用户进程设置正确的栈指针之前就到来了中断, 我们的系统还能够正确地进行中断处理吗?

留坑。

用户态和栈指针

> 一般来说, 处理器的特权级也是一种状态. 我们是否可以通过栈指针的值来判断当前位于用户态还是内核态?

留坑。

系统的复杂性(3)

> 我们把CTE重入问题作为状态机视角的一个练习, 请你来尝试分析上述代码在CTE重入的情况下存在什么问题.

上述代码在进入 `__am_irq_handle(c)` 的时候还没有对 `pp` 进行设置，比方将从用户态通过系统调用陷入内核态，此时 `pp == USER`，然后如果在系统调用中途发生了 CTE 重入，进入 `__am_asm_trap` 的时候，`pp` 仍然是 USER，会出错。

为了预防 CTE 重入导致的上述问题，我们要提前设置好 `pp`。因为 CTE 重入一定是从 CTE 进入 CTE，而 CTE 一定在内核态，所以直接将 `pp` 设为 KERNEL 即可。

临时寄存器的方案

> 同样是作为CTE的临时寄存器, mips32选择在GPR中分配出`k0`和`k1`, 而riscv32则是采用`mscratch`这个CSR寄存器. 这两种方案相比, 是否存在哪一种方案更优? 为什么?

留坑。

[二周目] Nanos-lite 与并发 bug

> 上文讨论并发的时候提到: 更一般地, 用户进程都会并发地执行系统调用, 操作系统还需要保证它们都能按照系统调用的语义正确地执行.
>
> 我们在PA3中知道`printf()`会通过`malloc()`申请缓冲区, 而`malloc()`又可能会执行`_sbrk()`, 通过`SYS_brk`陷入内核; 在上一个阶段中, 我们实现了支持分页机制的`mm_brk()`, 在必要的时候它会通过`new_page()`申请一页. 而仙剑和hello用户进程都会调用`printf()`, 使得它们可能会并发执行`SYS_brk`. 思考一下, 目前Nanos-lite的设计会导致并发bug吗? 为什么?

留坑。

补充

关于时钟中断的软件+4

时钟中断和 `yield` 的处理过程几乎完全相同。但需要注意的是，`yield` 中 mepc 保存的是当前指令 (ecall) 的 pc，而时钟中断由于放在 CPU 循环的 updatepc 之后，所以 mepc 保存的是下一条指令的 pc。因此时钟中断在操作系统层面不需要+4。

关于从 nterm 中执行其他程序时出错

笔者在实现了 `execve` 调用之后尝试从 nterm 中用命令 `pal --skip` 加载仙剑奇侠传并跳过片头动画，但触发了段错误。有效的 debug 路径如下：

```
发现pc是在内核区跑飞的 --> 在指令序列中寻找pc对应的指令，发现位于context_uload里面 --> 使用在各个位置插入assert(0)的方法来定位出错的具体语句，发现在完成protect()，切换到新进程的用户栈上，准备将参数搬上用户栈时发生段错误
```

nterm 的参数解析写在内建 shell 中，笔者在实现内建 shell 时将参数搬到了 `buildin-shell.c` 文件中定义的数组里。这些数组在编译是位于 nterm 进程的数据段。在 `context_uload()` 中完成 `protect()` 后，satp 切换到了新进程的页表。新进程的页表中没有 nterm 用户栈的映射信息，因此保存在 nterm 数据段的参数是取不到的。强行取就会触发段错误。但是我们必须在页表切换到新进程之后才能开始保存参数，这似乎是一个两难的问题。

笔者采取的解决方案是，进入 `context_uload()` 之后，先把所有的参数复制到内核代码创建的数组中，这些数组位于内核的数据段，是所有进程都可以访问的。这样切换到新进程的页表之后，再将已经保存在内核数据段的参数拷贝到新进程的用户栈上，就不会出问题了。

关于反复加载 bird 用户程序时出错

笔者给 bird 程序绑定了 esc 键用于退出，但笔者发现如果退出时再次加载 bird 程序会触发段错误，有效的 debug 路径如下：

```
发现pc是在用户区跑飞的 --> 使用strace打印执行过的所有系统调用 --> 发现最近一次使用的系统调用是sys_brk --> 检查sys_brk代码，思考第二次加载与第一次加载的不同
```

笔者发现关键点在于 max_brk 的维护上。loader 中将各个段加载到虚拟内存中，并根据加载的位置更新 max_brk。因此加载结束时 max_brk 总是能指向数据段之后第一个未被分配的地址。

```
for (int i = 0; i < ehdr->e_phnum; i ++)    if (phdr[i].p_type == PT_LOAD)    {
     fs_lseek(fd, phdr[i].p_offset, SEEK_SET);      fs_read(fd, data,
phdr[i].p_filesz);      uintptr_t _ed = memory_loader(&pcb->as,
phdr[i].p_vaddr, phdr[i].p_filesz, phdr[i].p_memsz);      if (_ed > pcb-
>max_brk) pcb->max_brk = _ed;    }
```

用当前 pcb 重新加载另一个用户进程时，一定要将 max_brk 清零，否则如果下一个进程的实际的 max_brk 小于前一个进程结束时的 brk，就会出现一段没有在页表中映射到的“真空地带”，在进程使用堆区的时候就会出发段错误。

关于实现内核栈切换之后时钟中断无法与 `yield()` 并存

笔者在实现内核栈切换之后发现如果既有 `yield()` 又有时钟中断会出现段错误。有效的 debug 路径如下：

```
发现pc是在内核区跑飞的 --> 使用itrace打印跑飞之前执行过的若干条指令 --> 发现在trap.S的代码执行了一半的时候又触发了时钟中断，出现了中断嵌套
```

经过对 `trap.S` 中代码的检查，发现了如下问题：

```
    STORE t0, OFFSET_CAUSE(sp)  STORE t1, OFFSET_STATUS(sp) STORE t2,
OFFSET_EPC(sp)    STORE t3, OFFSET_SP(sp)     addi t0, sp, CONTEXT_SIZE    csrr
t1, mscratch    xor t0, t0, t1    bnez t0, user_np    addi t2, x0, KERNEL
STORE t2, OFFSET_NP(sp)  j continueuser_np:  addi t2, x0, USER   STORE t2,
OFFSET_NP(sp)     # c->npcontinue:# set mstatus.MPRV to pass difftest    li a0,
(1 << 17)    or t1, t1, a0    csrw mstatus, t1
```

最后一段有三条用于 pass difftest 的指令，这里的指令默认 t1 中仍然存储的是 `STORE t1, OFFSET_STATUS(sp)` 中的 mstatus 的值（在原本的 `trap.S` 的代码中它们是连在一起的）。但笔者现在在中间加入了一些判断状态的汇编语句，破坏了 `t1` 寄存器的值，当 `t1` 的值的 MIE 位正好是 1 的时候，执行完最后的 `csrw mstatus, t1` 就会神不知鬼不觉地打开中断，从而导致中断嵌套。

修复的方法很简单：只要将 pass difftest 的三条语句移动到 `STORE` 的下面即可。

关于实现内核栈切换之后无法通过 OJ

笔者在实现了内核栈切换之后发现无法通过 OJ 的 easy test。这是笔者调过的最痛苦的 "bug"，因为本地没有任何的错误，无法找到任何不对劲的地方，只能虚空调 bug。后来笔者通过在各个函数中加入 `assert(0)` 的方法来测试 OJ 到底是如何评测的，惊奇地发现 OJ 使用了 `kcontext()` 但没有使用 `context_kload()` ，即 OJ 只是创建了一个内核线程上下文，但没有使用进一步封装的 `context_kload()` 将其保存在某个具体的 pcb 的内核栈上。

笔者默认创建上下文会调用更高层的 `context_kload()` ，不会绕过它直接调用底层的 `kcontext()` ，因此在实现时并没有在 `kcontext()` 中完成所有的上下文信息的设置，c->sp 的设置被放在了 `context_kload()` 里面。因此单纯调用 `kcontext()` 获得的上下文结构体缺少了 c->sp，栈切换时 sp 寄存器自然会跑飞。将这一初始化挪到 `kcontext()` 里面完成后即可通过 OJ。

这一教训告诉我们必须仔细阅读讲义的 API 定义。讲义中关于 `kcontext()` API 的定义是：

> “其中`kstack`是栈的范围, `entry`是内核线程的入口, `arg`则是内核线程的参数. 你需要在`kstack`的底部创建一个以`entry`为返回地址的上下文结构(目前你可以先忽略`arg`参数), 然后返回这一结构的指针.”

因此确实应在在 `kcontext()` 内完成所有创建工作。

编写不朽的传奇

思考题

笔者尚未完成该章节的选做内容，故此部分的思考题暂时留坑。

补充

关于使用 `F1` `F2` `F3` 切换时画面互相覆盖

由于没有用于调度窗口的进程，在进程切换的时候画面会互相覆盖，非常难看。笔者采用的暂时性的解决方案是在进程切换的时候强行刷新一次 VGA，将所有像素填成黑色。这个方案对于 bird，nterm 等整个屏幕都在不断更新的应用比较友好，但对于 pal，nslider 等没有变化的区域屏幕不会更新的应用不太友好——屏幕上没有更新的部分会一直是黑的，除非对应用进行操作使其刷新屏幕。笔者认为如果将整个屏幕的像素信息加入上下文，在恢复上下文的时候将整个画面恢复会有比较好的效果。但这样上下文结构体会极其臃肿。